A Learned Warm Start for Phase-Fixed Weighted Gerchberg-Saxton Does Not Beat an Early Phase Freeze: A Measured Negative Result on Holographic Optical Trap Arrays

Felipe Santibáñez-Leal
CAOS open-research programme, Santiago, Chile
fsantibanez@gmail.com

Preprint · v0.01, 3 November 2026 · Licence CC BY 4.0
CAOS open-research programme · CEFOP_DinHot, holographic optical trap studio
Version DOI 10.5281/zenodo.24022471 · concept DOI (always latest) 10.5281/zenodo.24022470
Not peer reviewed. Source code, data and computational records:
github.com/fsantibanezleal/CEFOP_DinHot

Abstract

Phase-fixed weighted Gerchberg–Saxton (WGS) is the practical method for large arrays of optical traps, and its cost is the number of iterations before the trap intensities are uniform. We tested whether a small permutation-equivariant network that predicts the initial phase of every trap (a learned warm start, 80 899 parameters) lets phase-fixed WGS reach a relative standard deviation of 1 percent in at least two times fewer iterations than a random start on geometries held out from training, with the final efficiency within 1 percent. On 66 held-out geometries (60 generated, 6 benchmark; 3 seeds, a budget of 60 iterations) the network start needs a median of 8 iterations (bootstrap 95% interval 7 to 17) against 14 from a random start, a factor of 1.75 (interval 0.85 to 2), below the factor of two required, and the final efficiency differs by 4.3 percent, above the 1 percent allowed. A control with no network (a random start with the same early freeze of the phase) needs 7.5 iterations (factor 1.87, interval 1.38 to 2.33), so the gain belongs to the freeze index and not to the network; the network with the engine's own freeze index needs 14 iterations, as the random start does. The shorter freeze has a price: 32.8 percent of its runs never reach 1 percent within 60 iterations, against 6.1 percent for the default. In a sweep of the freeze index the median iterations to 1 percent lie 2 to 4 above the freeze index for freeze indices 4 to 12 and 6.5 above at 3, where 41.1 percent of the runs never converge; at 1 and 2 the median is the cap of 61 iterations (76.7 and 61.1 percent of the runs never converge); the phases of a converged run save 1 iteration at the default freeze and lower the median from 11.5 to 5.5 at a freeze of three (ten geometries, one seed). The model is not released. The results hold on a scalar Fourier model and one trained network, not on hardware.

Keywords: holographic optical tweezers, weighted Gerchberg–Saxton, learned initialization, warm start, negative result.

1 Introduction

The problem. A phase-only spatial light modulator (SLM) in the pupil plane of a lens turns a computed phase pattern into an array of optical traps in the focal plane. The arrays now reach thousands of sites: the 256-atom quantum simulator of Ebadi et al. [12] used arrays of roughly one thousand tweezers computed with phase-fixed weighted Gerchberg–Saxton (WGS), and the 6100-atom array of Manetsch et al. [13] spans about twelve thousand sites with a trap-depth spread of 11.4 percent after WGS. The spread matters because the trap depth is proportional to the intensity, so every percent of intensity spread is a percent of depth spread across the atoms [32]. The method that delivers percent-level uniformity at this scale is phase-fixed WGS of Kim et al. [6], which freezes the focal-plane phase after a few iterations so that the weights of the update remain effective. Its cost is iterations: each one is a pair of Fourier-type transforms of the full SLM field, and the number of iterations before the intensities are uniform decides how fast a new geometry can be

displayed, how many holograms a rearrangement sequence can afford and how much computation a browser or an embedded controller must carry.

The idea. Learning-based methods have been applied to hologram generation as replacements of the iteration: DeepCGH [22], tensor holography [23], and a generative network for tweezer arrays [25] produce a phase directly. Direct generation trades accuracy for speed: the network of Ren et al. [25] reports about 10 percent variation of the trap depth on about 20 sites, which is far from the 1.4 to 1.6 percent that WGS with camera feedback reaches [6, 10]. A different use of the same network is to give the iteration a better beginning, a warm start, as has been proposed for fixed-point solvers in optimization [26]. The iterative method then keeps its guarantees and its final accuracy, and the network can only save iterations. The warm start we study predicts one initial phase per trap from the list of trap coordinates and weights with a permutation-equivariant set network; the superposition hologram built with those phases replaces the random-phase start of phase-fixed WGS.

The gap. A search of the literature for this combination (Section 2) found learned networks that generate a hologram and learned warm starts for generic fixed-point solvers, and found no work that initializes WGS with a network or compares such a start with a random start on the same task. The question was therefore open, with an explicit risk: the random-phase start of WGS is already reliable, so the gain may be small.

What we did. We implemented the warm start, trained it without labels through a differentiable copy of the phase-fixed WGS iteration, and fixed the acceptance test before training: on geometries held out from training, the median number of iterations to a relative standard deviation of 1 percent must be at least two times lower than from a random start, the final efficiency and uniformity must agree within 1 percent, and the network must fit in 1 MB. We then measured it. The network start misses the first two clauses (a factor of 1.75 on the median iterations, final efficiency 4.3 percent apart). The quantity that explains the gain turned out to be a setting of the iteration itself: the index at which phase-fixed WGS freezes the focal phase, n_{fix}. A random start with the same early freeze ($n_{\mathrm{fix}} = 3$ instead of the engine default of 12) needs 7.5 iterations against 8 for the network, so the network contributes nothing measurable beyond the freeze.

Contributions. We state them as findings, not as a method that works.

1. A learned warm start for phase-fixed WGS: the architecture, the unsupervised training through an unrolled differentiable copy of the iteration, and a transparent acceptance test fixed in advance (Sections 4 and 6).

2. A measured negative result with its uncertainty on 66 held-out geometries, run with the engine of the product on the same data as the baselines (Section 7).

3. A four-arm ablation that separates the network start, the early phase freeze and their combination, with paired sign tests and bootstrap intervals (Section 8); it attributes the gain to the freeze.

4. An analysis of why a start cannot help much: four propositions (the invariances of the iteration and of the network, the dependence of the run on the start through the freeze index, and an accounting bound on the weights) and a conjecture on the iteration count; a sweep of the freeze index and an oracle start from converged phases (Sections 5 and 8).

5. The price of the shorter freeze, which the median hides: the share of runs that never reach 1 percent within the budget rises from 6.1 to 32.8 percent.

Main result. On held-out geometries a network start with an early freeze reaches 1 percent of relative standard deviation at a median of 8 iterations (95% interval 7 to 17), against 14 from a random start with the default freeze (factor 1.75, interval 0.85 to 2, below the factor of two required); a random start with the same early freeze reaches it at 7.5 (6 to 10.5). The claim that a learned start accelerates phase-fixed WGS is not supported for this trained network on this model (one network, one training seed), and the network is not released. The oracle start of Section 8.3 shows a headroom at an early freeze that this network does not reach.

Structure. Section 2 places the work among the published methods, Section 3 states the problem, Section 4 derives the method, Section 5 analyses it, Section 6 states the hypotheses and the protocol, Sections 7 and 8 report the measurements, and Sections 9 and 10 discuss them and their limits. Sections 11 and 12 give the commands that rebuild every number and the conclusions.

2 Related work

We group the published methods by what they optimize and what they assume, and state for each what it gives and where it stops. The characterizations are those of the cited works; where a work was known to us through its abstract only, we say what the abstract states and no more.

2.1 Iterative and analytic hologram design for trap arrays

The iteration of Gerchberg and Saxton [1] recovers a phase from two intensity measurements by alternating between the two planes and replacing the amplitude in each by the measured one; it became the standard way of computing a phase hologram for a target intensity. Di Leonardo, Ianni and Ruocco [2] compare, for arrays of optical traps in the focal plane of a lens, the random superposition of single-trap holograms (the SR hologram), the Gerchberg–Saxton iteration and a weighted form in which the amplitude imposed on every trap is multiplied by a weight that is updated to reduce the deviation of the trap amplitude from the average amplitude. The weighted form equalizes the trap intensities, and the SR hologram, one pass with one random phase per trap, is the usual start of the iterations. Bianchi and Di Leonardo [3] give GPU implementations of the same two algorithms and report their timings, which established that the iteration can be run interactively. Whyte and Courtial [5] extend the iteration to three-dimensional targets by propagating the target plane by plane. All of these start from random trap phases, and all treat the start as a nuisance whose effect averages out: the iteration is run from a random start until the metric is reached.

Polin et al. [4] take a different route for the same task, direct search and simulated annealing on the phase of individual pixels with a cost that trades efficiency for uniformity; for 119 quasi-periodic traps they report a brightness variation of the iterative holograms above 50 percent against below 5 percent for direct search. The method does not need a start of any kind, but it is sequential and cost grows with the number of pixels and phase levels, so it does not meet the iteration budgets of large arrays.

2.2 Weights, phase freezing and camera feedback

The two ingredients that make weighted GS reach percent-level uniformity on large arrays are a weight that is updated multiplicatively and, after a few iterations, a frozen focal-plane phase. Kim et al. [6] show that replacing the SLM-plane amplitude by the laser amplitude perturbs the focal phase and makes the weights less effective, and that fixing the phase after an initial number of iterations removes the perturbation; on 1500 foci with twelve iterations before the freeze they obtain a modulation efficiency of 91.2 percent and a non-uniformity below 0.5 percent, and with three camera-corrected steps a non-uniformity of 1.59 percent. Nogrette et al. [10] apply a closed-loop rescaling of the target with a gain of about 0.7 and bring the rms trap-depth non-uniformity of about one hundred traps from 19 to 1.4 percent in 20 iterations. Chew et al. [11] report, in an abstract, an intensity relative standard deviation of 0.3 percent, shape variations below 0.5 percent and positioning errors within 70 nm for holographic arrays of an SLM, with a series of optimization methods fed by precise measurement. The experiments with the largest arrays use this family: Ebadi et al. [12] compute their holograms with phase-fixed WGS and Manetsch et al. [13] weight it with the measured loading probability. In every one of these works the start is a random-phase superposition, and the freeze index is a parameter whose value, twelve in [6], is taken as a given.

2.3 Gradient methods and models of the device

Harte et al. [14] minimize a cost on the focal intensity by conjugate gradients with the gradient in closed form through the fast Fourier transform; they report errors of 0.07 percent on shaped targets after thousands of iterations and note that the method needs more iterations than mixed-region amplitude freedom but fewer random restarts. Bowman et al. [15] extend the conjugate-gradient approach to control of both phase and amplitude with a phase-only SLM. Chakravarthula et al. [16] treat the hologram as the minimizer of a loss through any differentiable propagation model with Wirtinger derivatives, and Schroff et al. [17] put a model of the pixel crosstalk of the SLM inside such a loop. These methods can reach lower errors than the Gerchberg–Saxton family and can include a device model, at the price of a higher cost per hologram; none of them is a warm start, and each of them is itself started from a random or a Gerchberg–Saxton phase, so a better start would help them as well.

2.4 Shaped targets: mixed-region freedom and optimal transport

For continuous targets, Pasienski and DeMarco [18] let the amplitude float in a noise region (mixed-region amplitude freedom, MRAF), and Holland et al. [19] compare rounding, error diffusion and MRAF for large periodic arrays on a phase SLM and on a binary mirror device. The one published place where the start of the iteration is treated as the object of the method is optimal transport. Swan et al. [20] solve a quadratic-cost optimal-transport problem between the input and the target intensities and use the resulting phase as the start of GS or MRAF, which in their six targets lowers the MRAF error from the range of 1.5 to 3.9 percent to 0.17 to 0.96 percent and raises the efficiency; Torchylo et al. [21] reduce the memory of the computation to a linear order with a separable cost, so a 1024×1024 problem takes 8.5 seconds on a GPU. These starts are computed from the target by an optimization, not predicted by a trained model, and they address shaped targets; the point-trap array is not their subject.

2.5 Learned holograms and learned warm starts

DeepCGH [22] trains a convolutional network without labels by minimizing a differentiable accuracy of the reconstruction and reports volumetric holograms at least ten times faster than Gerchberg–Saxton and 9 to 43 percent more accurate on its metric. Tensor holography [23] reports a network of under 620 KB that computes holograms at video rate (the abstract is what we read). Neural holography [24] calibrates a differentiable propagation model with a camera in the loop. Ren et al. [25] train a U-Net on 2000 hologram pairs produced by Gerchberg–Saxton to generate the phase of tweezer arrays in 0.16 s against about 6 s for the iteration at a comparable signal-to-noise ratio, with about 10 percent trap-depth variation on about 20 sites. All four replace the iteration with a network; none uses the network to start an iteration and none compares with weighted GS on the uniformity of hundreds of traps.

Outside holography, learned warm starts for fixed-point solvers are an established idea. Sambharya et al. [26] map the parameters of a problem to a warm start with a neural network followed by a fixed number of iterations of the solver, train it end to end on the downstream loss, and report a significant reduction in the number of iterations on control, statistics and signal-processing problems. Our network uses the same recipe, an unrolled solver as the training loss, on a solver (phase-fixed WGS) that differs in one respect from those of [26]: its state includes weights that restart at one, and a phase that is frozen at an index of the user's choice.

2.6 Sequences and rearrangement

The holograms of rearrangement sequences are computed in large numbers, which is where a saving of iterations would count most. Kim et al. [7] adapt the iteration to atom rearrangement with zero padding and weighted feedback, Lee et al. [8] choose the assignment of atoms to sites by the Hungarian method, and Knottnerus et al. [9] interpolate position and optical phase between the first and last hologram to display new holograms every few milliseconds with a success probability of 0.996 per rearrangement cycle. The last method avoids the iteration for the intermediate frames by interpolating the phases of the endpoint holograms, a different way of re-using a computed start.

A warm start would matter most in these sequences, where thousands of holograms are computed one after the other and each begins from the previous one. In that setting the natural start of a new hologram is the preceding hologram, not a random phase, which is why the sequence methods already re-use phases; the question for a learned warm start is therefore the case of a hologram computed for a new geometry, with no predecessor to start from, which is the case we test.

2.7 Position of the proposed method

The proposed method sits between the learned generators and the iterative methods. Like [22, 25] it trains a network on random geometries to learn a map from the trap list to phases; unlike them the output is not the hologram but the initial phase of an iteration that keeps its accuracy, so the final uniformity of phase-fixed WGS [6] is retained by construction. Like [26] it is a learned warm start trained through the unrolled solver; unlike it, the solver has a frozen phase and a weight state that restart. Like [20] it treats the start as a design object; unlike it, the start is predicted in one forward pass from the trap list and does not solve a transport problem. Among the works above, only [26] and [20] treat the initialization of an iterative solver as the quantity to improve, and neither involves a solver with a frozen phase. The experiments that follow

ask whether this position is worth anything in iterations, and the answer, with a control that none of the cited works needed, is no.

3 Problem formulation

3.1 Forward model

The SLM has $N \times N$ pixels with $N = 256$ and pitch p, and lies in the pupil plane of a lens of focal length f illuminated at wavelength λ by a Gaussian beam of amplitude A_j normalised so that $\sum_j A_j^2 = N^2$. We use the scalar Fourier model of Di Leonardo et al. [2]. A trap $m = 1, \dots, M$ at lateral position (x_m, y_m) and depth z_m in the focal region, with a target weight $w_m > 0$, sees a field and an intensity

$$V_m(\phi) = \frac{1}{N^2} \sum_j A_j \, e^{\,i(\phi_j - \Delta_j^m)}, \qquad I_m = |V_m|^2, \qquad \Delta_j^m = \frac{\pi z_m}{\lambda f^2}\,(x_j^2 + y_j^2) + \frac{2\pi}{\lambda f}\,(x_j x_m + y_j y_m), \tag{1}$$

where ϕ_j is the phase of pixel j at position (x_j, y_j) and Δ_j^m the single-trap delay (a prism for the lateral position and a lens for the depth). The adjoint operation $F_j = \sum_m c_m \, e^{+i\Delta_j^m}$ back-projects trap coefficients to the SLM plane; every weighted Gerchberg–Saxton update is the argument of such a sum. The lens term is written in units of the Rayleigh range that the SLM aperture focuses, $z_{\mathrm{R}} = \lambda f^2/(\pi R^2)$ with $R = Np/2$, so that the quadratic phase at the aperture edge equals z_m/z_{R} radians, and the lateral position in focal bins of the zero-padded transform, $\lambda f/(\mathrm{pad}\, Np)$. In these units the model does not depend on λ, f or p, which is why the network of Section 4 can take the trap list in bins and Rayleigh ranges as its only input.

3.2 Metrics and the quantity to minimize

The quality of a hologram is read on the trap intensities divided by the target weights, $r_m = I_m / w_m$. The efficiency, the relative standard deviation and the uniformity are

$$\eta(\phi) = \sum_{m=1}^{M} I_m, \qquad \sigma(\phi) = 100\,\frac{\mathrm{std}_m(r_m)}{\mathrm{mean}_m(r_m)} \;\; [\%], \qquad u(\phi) = 1 - \frac{\max_m r_m - \min_m r_m}{\max_m r_m + \min_m r_m}, \tag{2}$$

with the population standard deviation. The efficiency is the fraction of the input power in the peaks of the traps; σ is the quantity a camera-corrected experiment cares about and the one the methods of the family drive down. An iterative method produces a sequence of phases $\phi^{(k)}$, $k = 1, 2, \dots$, and the cost of the method on an instance is the number of iterations before the spread falls to a threshold,

$$k_1 = \min\{\, k \geq 1 : \;\; \sigma(\phi^{(k)}) \leq 1 \,\}, \tag{3}$$

which we take as $60 + 1$ when the threshold is not reached within the budget of 60 iterations. The threshold of 1 percent is the level the acceptance test of the product sets for a hologram before camera feedback, a spread that the camera-corrected experiments [6, 10] reach at 1.4 to 1.6 percent after feedback. The final values of η and σ at the end of the budget are read as well, because a start that saves iterations by degrading them has not saved anything.

3.3 The warm-start problem

Let $\mathcal{A}$ be an iterative method that, given a start phase $\phi^{(0)}$ and a trap set $\mathcal{T} = \{(x_m, y_m, z_m, w_m)\}_{m \leq M}$, produces $\phi^{(k)}$. The random start of the literature draws one phase θ_m per trap uniformly in $[0, 2\pi)$ and builds the superposition hologram of Section 4. A warm start is a map $g_\vartheta : \mathcal{T} \mapsto (\theta_1, \dots, \theta_M)$ with parameters ϑ, learned from a distribution $\mathcal{D}$ of trap sets, such that the expected cost

$$J(\vartheta) = \mathbb{E}_{\mathcal{T} \sim \mathcal{D}} \; \mathbb{E}_{\mathrm{seed}} \big[\, k_1\big(\mathcal{A}(\phi^{(0)}_{g_\vartheta(\mathcal{T})}, \mathcal{T})\big)\big] \tag{4}$$

is lower than with random phases. The quantity of interest is the ratio of the median k_1 of the random start to that of the learned start on trap sets that are not in the training distribution, so that a warm start that only memorizes the training geometries gains nothing. The acceptance criterion fixed before training asks for a ratio of at least two with the final efficiency and uniformity within 1 percent.

Symbol	Meaning
N, p, f, λ	SLM pixels per side, pixel pitch, lens focal length, wavelength
A_j, ϕ_j, Δ_j^m	input amplitude, phase of pixel j, single-trap delay of trap m at pixel j
M, w_m, $a_m = \sqrt{w_m}$	number of traps, target weight, target amplitude
V_m, $I_m = \lvert V_m \rvert^2$	trap field and intensity; $r_m = I_m / w_m$
η, σ, u	efficiency, relative standard deviation (%), uniformity
k_1	iterations to $\sigma \leq 1\%$, with the value $60 + 1$ when never reached
θ_m, $\phi^{(0)}$	initial trap phase and the superposition start hologram built from it
g_k, $B_k(m) = \lvert V_m \rvert / a_m$	weights of the update at iteration k and the relative trap amplitude
ψ_k, n_{fix}	focal-plane phase used at iteration k; index at which it is frozen
g_ϑ	the warm-start network with parameters ϑ
S_k, s_k	log-spread of the weights after iteration k and the allowance of the iteration

Table 1: Notation: the symbols of the forward model, the metrics, the iteration of phase-fixed WGS and the warm-start network used in Sections 3 to 8.

The expectation of Eq. (4) is over two sources of randomness, the geometry and the seed of the random phases that the baseline uses, and the learned map is deterministic, so its run varies only with the geometry. The comparison therefore pairs each geometry and seed across arms: the random start of the baseline and the network start see the same trap set, and the baselines with different freeze indices see the same random phases. A warm start is useful when it lowers the median of k_1 over the geometries; it is harmful when it raises the share of runs that do not converge, a quantity that the median of k_1 alone cannot show and that we therefore report next to it.

3.4 Assumptions and notation

We assume (i) the scalar Fourier model of Eq. (1), ideal pixels (no crosstalk, quantization or flicker), because the question concerns the iteration and not the device; (ii) point traps without vortex charge, with at most 1024 traps, the limit of the network input; (iii) a Gaussian input beam, one lens and an SLM of 256^2 pixels with a zero padding of two for the generated geometries; and (iv) a budget of 60 iterations per run. Assumption (i) is the model of the engine of the product on which every method of the study runs; the effect of a real device on the question is outside the measurements (Section 10). Two further conventions matter for what follows. The trap weights w_m enter the iteration only through the target amplitudes $a_m = \sqrt{w_m}$ and through the division of the intensities by w_m in the metrics, so a set with unequal weights is a different problem from a set with equal weights on the same positions. The spread σ is computed on the intensities divided by the weights, which is the quantity a user of the array wants to equalize. Table 1 lists the symbols.

4 Proposed method: a learned warm start for phase-fixed WGS

4.1 Idea

Phase-fixed WGS [6] starts from the superposition hologram of random trap phases, whose field at the traps has random phases that the first iterations must organize. The idea of the proposed method is to replace those random phases by phases that a network predicts from the geometry of the trap set, so that the first field the iteration sees is already close to the phase pattern it would converge to. Two properties make the idea attractive on paper. First, the network is only a start: whatever it predicts, the iteration that follows is the unchanged phase-fixed WGS, so the accuracy it reaches in the limit of many iterations is that of the published method, and a bad prediction costs iterations and nothing else. Second, the iteration is self-correcting, so the network does not need to be precise: a coarse prediction of the phase law of the array (the quadratic and periodic phase patterns that regular arrays converge to) may be enough to save iterations. The expected benefit is therefore bounded by the number of iterations the random start wastes before the field organizes. Whether that number is large is the empirical question the paper answers.

Figure 1 shows the pipeline: the trap list goes through the network, the predicted phases build the start hologram, and the unchanged iteration runs from it; the weights of the iteration are not an output of the network.

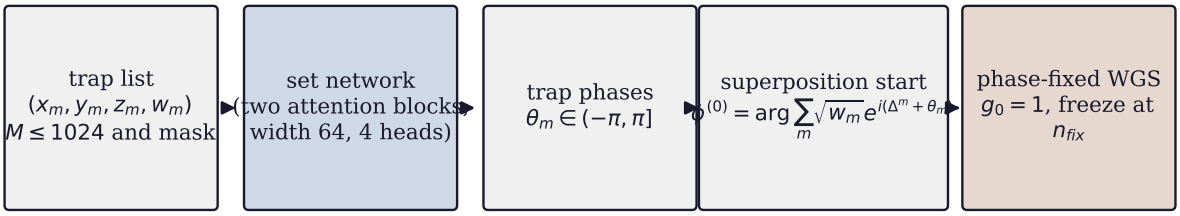


Figure 1: The learned warm start: a trap list of at most 1024 traps with a mask enters a permutation-equivariant set network that predicts one phase per trap; the superposition hologram built from those phases starts phase-fixed WGS, whose weights restart at one; training minimizes the log spread along an unrolled copy of the iteration, without labels.

4.2 The start hologram

Given predicted phases θ_m, the SLM start is the superposition of the single-trap holograms with those phases, the form of the SR hologram of [2] with learned instead of random phases:

$$\phi_j^{(0)} = \arg \sum_{m=1}^{M} a_m \, e^{\,i(\Delta_j^m + \theta_m)}, \qquad a_m = \sqrt{w_m}. \tag{5}$$

This is the back-projection of the coefficients $a_m e^{i\theta_m}$ of Section 3.1, and it is the same operation the random start performs with θ_m drawn from the portable random generator of the engine. The start field is $V_m^{(0)} = V_m(\phi^{(0)})$ from Eq. (1). When the traps are well separated the cross terms between traps are small and $\arg V_m^{(0)} \approx \theta_m$, so θ_m is, to a first approximation, the phase the iteration sees at trap m in its first step. When the traps are dense the cross terms are not small, and $\arg V_m^{(0)}$ differs from θ_m; this is the case of every geometry in the held-out set (at least 40 traps).

The start hologram is the only place where the network output meets the engine, and its form was chosen so that a network that predicts the right phases reproduces exactly what a converged run would hold: the phases of the converged field at the traps, back-projected with the target amplitudes. The same function builds the oracle start of Section 6.5, with the phases of a converged run in place of the network phases, so that the two starts differ only in where the phases come from and the comparison is not confounded by a different construction of the hologram.

4.3 The iteration the start enters

Phase-fixed WGS keeps one weight $g_{k,m}$ per trap and one focal phase $\psi_{k,m}$ per trap. With $B_k(m) = |V_m^{(k)}|/a_m$ the trap amplitude relative to its target, the iteration for $k \ge 1$ is

$$g_{1,m} = 1, \qquad g_{k,m} = g_{k-1,m} \, \frac{\langle B_{k-1} \rangle}{B_{k-1}(m)} \quad (k \ge 2), \tag{6}$$

$$\psi_{k,m} = \arg V_m^{(k-1)} \quad (k \le n_{\mathrm{fix}}), \qquad \psi_{k,m} = \psi_{n_{\mathrm{fix}},m} \quad (k > n_{\mathrm{fix}}), \tag{7}$$

$$\phi_j^{(k)} = \arg \sum_{m=1}^{M} g_{k,m} \, a_m \, e^{\,i(\psi_{k,m} + \Delta_j^m)}, \tag{8}$$

where $\langle \cdot \rangle$ is the mean over the traps and n_{fix} the freeze index; the engine default of the product is $n_{\mathrm{fix}} = 12$ after Kim et al. [6]. The weights update multiplicatively and carry the memory of all earlier corrections; the focal phase follows the field for the first n_{fix} iterations and is then fixed, which removes the focal-phase perturbation that re-imposing the laser amplitude at the SLM would otherwise introduce. The start enters in one place only: the field $V^{(0)}$ of Eq. (5), which gives the phases $\psi_1 = \arg V^{(0)}$ of the first iteration. The

weights restart at one whatever the start is. The relative amplitude B_k is the field amplitude $|V_m|$ divided by the target amplitude a_m, as in weighted GS [2]. This is the structural fact on which the theory of Section 5 rests: a warm start can only change the iteration through the phases of the first n_{fix} steps.

4.4 The network

The network is a permutation-equivariant set transformer over the trap list (Set Transformer [27], Deep Sets [28]). Each trap m is described by its position in focal bins $(\hat{x}_m, \hat{y}_m) = (x_m, y_m)/128$, its depth in Rayleigh ranges $\hat{z}_m = z_m/(4z_{\mathrm{R}})$ and its weight, and receives a vector of 29 features: the normalized coordinates, $\hat{z}_m$, $w_m - 1$, $\varrho_m^2 = \hat{x}_m^2 + \hat{y}_m^2$, and Fourier features of the position at six octaves [29],

$$\mathbf{f}_m = \big[\hat{x}_m, \hat{y}_m, \hat{z}_m, w_m - 1, \varrho_m^2,\ \sin(\pi\nu\hat{x}_m), \cos(\pi\nu\hat{x}_m), \sin(\pi\nu\hat{y}_m), \cos(\pi\nu\hat{y}_m)\big]_{\nu\in\{1,2,4,8,16,32\}}. \tag{9}$$

Two layers embed $\mathbf{f}_m$ to width $d = 64$. Two blocks of attention follow, each with 4 heads of width 16, a residual connection, and a feed-forward layer of width 128, with the padded traps removed by an additive bias of -10^4 on the attention logits:

$$\mathbf{h} \leftarrow \mathbf{h} + W_o\,\mathrm{softmax}\Big(\frac{QK^\top}{\sqrt{d_h}} + \mathbf{b}\Big)V, \qquad \mathbf{h} \leftarrow \mathbf{h} + W_2\,\mathrm{ReLU}(W_1\mathbf{h}), \qquad b_{jm} = -10^4\,(1 - \mathrm{mask}_m), \tag{10}$$

where Q, K, V are linear maps of $\mathbf{h}$ and $d_h = 16$; the output projections W_o and W_2 start at zero so that the untrained network is the embedding alone. No layer normalization is used, so that the exported graph contains only operators that a browser inference runtime provides. With $\bar{\mathbf{h}}$ the mean of $\mathbf{h}$ over the valid traps, the predicted phase of trap m is

$$\theta_m = \mathrm{wrap}\Big(\mathrm{MLP}\big([\mathbf{h}_m, \bar{\mathbf{h}}]\big) + \big(c_0 + 20\,\mathbf{w}_c^\top\bar{\mathbf{h}}\big)\,\varrho_m^2\Big), \qquad \mathrm{wrap}(t) = \mathrm{atan2}(\sin t, \cos t), \tag{11}$$

where the second term is a set-dependent quadratic (Fresnel-like) phase law with a learned scalar c_0 (initialized at 80) and a learned vector $\mathbf{w}_c$; it lets the network write the chirp that a regular array's converged phases follow without having to represent hundreds of radians with an MLP. The network has 80 899 parameters and its ONNX export holds 338 987 bytes. The value of c_0 after training is the initial one to within a fraction of a percent, so the learned part is the correction on top of the chirp.

4.5 Training without labels

Converged WGS phases are not unique: the global phase is free and the many fixed points of the iteration give different phases for the same geometry, so a regression onto converged phases would have an ambiguous target. We train instead through the iteration. A differentiable copy of Eqs. (6)–(8) (separable point-trap form in single precision, checked against the engine's phase-fixed WGS in the repository's tests) is unrolled for $K = 10$ iterations from the start of Eq. (5), and the loss is the weighted mean of the log of the squared spread:

$$\mathcal{L}(\vartheta) = \frac{1}{\sum_{k=0}^{K}\omega_k}\sum_{k=0}^{K}\omega_k\ \mathbb{E}_{\mathcal{T}\sim\mathcal{D}}\Big[\log\big(\sigma_k^2 + 0.01\big)\Big], \qquad \omega_0 = 0.3,\ \omega_{k\geq 1} = 1, \tag{12}$$

with σ_k the spread (in percent) after unrolled iteration k and σ_0 that of the start. The constant 0.01 saturates the loss below a spread of 0.1 percent, so that the network is not rewarded for precision the iteration reaches by itself; the log compresses the large spreads of the first iterations. The unrolled iteration uses the freeze index $n_{\mathrm{fix}} = 3$, chosen because an earlier trial at $n_{\mathrm{fix}} = 12$ with 14 unrolled iterations produced a start that reached about the random level. The global phase is invariant by construction because the loss reads intensities only. The trap sets are drawn on the fly from seeded generators: square and hexagonal lattices, rings with an optional concentric ring, random sets with a minimum spacing, stacks of two to four planes and random three-dimensional sets, with the proportions 0.25, 0.15, 0.15, 0.2, 0.15, 0.1, optional position jitter and random weights in $[0.5, 2]$ for 30 percent of the sets; sets of fewer than 40 traps are discarded with probability 0.6 because a sparse array reaches 1 percent at the first iteration from any start. The generators contain no quasi-periodic family, so the benchmark geometry of that kind is unseen.

4.6 Algorithms and parameters

Algorithm 1 is the method as run by the engine; Algorithm 2 is the offline training. The inference algorithm has two branches that matter for the interpretation of every measurement below: when a network is released

Algorithm 1 Learned warm start for phase-fixed WGS (inference and run).

Input: trap set $\mathcal{T} = \{(x_m, y_m, z_m, w_m)\}$, network g_ϑ (or none), freeze index n_{fix}, budget $K_{\max}$, seed
Output: SLM phase $\phi^{(K)}$ and the history $\sigma(\phi^{(k)})$

convert $\mathcal{T}$ to bins and Rayleigh ranges, pad to 1024 traps with a mask
if a network is released **then** $\theta \leftarrow g_\vartheta(\mathcal{T})$
else $\theta_m \sim U[0, 2\pi)$ from the seeded generator
end if
$\phi^{(0)} \leftarrow \arg \sum_m a_m e^{i(\Delta^m + \theta_m)}$ ▷ Eq. (5)
$V^{(0)} \leftarrow \text{project}(\phi^{(0)})$; $g_1 \leftarrow \mathbf{1}$
for $k = 1, \ldots, K_{\max}$ **do**
  if $k \geq 2$ **then** $g_k \leftarrow g_{k-1} \langle B_{k-1} \rangle / B_{k-1}$
  end if ▷ Eq. (6)
  $\psi_k \leftarrow \arg V^{(k-1)}$ if $k \leq n_{\text{fix}}$, else $\psi_{n_{\text{fix}}}$ ▷ Eq. (7)
  $\phi^{(k)} \leftarrow \arg \sum_m g_{k,m} a_m e^{i(\psi_{k,m} + \Delta^m)}$; $V^{(k)} \leftarrow \text{project}(\phi^{(k)})$ ▷ Eq. (8)
end for
return $\phi^{(K_{\max})}$ and the history of σ, η, u

Algorithm 2 Unsupervised training of the warm-start network through the unrolled iteration.

Input: generator of trap sets $\mathcal{D}$ with a seed, steps T, batch B, unroll K, freeze index n_{fix}
initialize ϑ from the initialization seed; AdamW with a warm-up and a cosine schedule
for $t = 1, \ldots, T$ **do**
  draw B trap sets, pad and mask; $\theta \leftarrow g_\vartheta(\mathcal{T})$
  run the unrolled iteration for K steps from the start of Eq. (5), recording $\sigma_0, \ldots, \sigma_K$
  $\mathcal{L} \leftarrow$ Eq. (12); clip the gradient norm to 1; update ϑ
end for
return ϑ ▷ evaluated on held-out geometries before any release

the phases come from it, and when none is released the phases are drawn from the seeded generator, so that the same function answers in both cases and the baseline is not a different code path. The training algorithm sees only trap sets and the differentiable iteration; it never reads a held-out geometry and never reads a converged phase, which is what makes it unsupervised in the sense of the problem statement.

Table 2 lists every parameter of the method with its value and where the value comes from. Values that are choices of the product, not of a source, are marked as such. The architecture was sized to the limit the product sets for a model that a browser must download and run, at most 1 MB as an exported graph, and the training budget to what a single laptop GPU delivers in under two hours; neither was tuned against the held-out set, which training never reads. The freeze index of the training unroll, $n_{\text{fix}} = 3$, is the one parameter that departs from the published default, and it is the parameter that the ablation of Section 8 isolates.

4.7 Complexity and implementation

The attention of a block over M traps costs $M^2 d$ multiply-adds for $QK^\top$ and the same for the weighted sum, that is $2M^2 d$ per block, so inference is $O(LM^2 d)$ for L blocks, plus $O(Md^2)$ for the linear layers; the full attention is quadratic in the number of traps, not linear. Counting the multiply-adds of the attention products and of one iteration of the point form of phase-fixed WGS (a projection and a back-projection, each a separable product of $N^2 M$ terms),

$$C_{\text{net}} \approx 2\, L\, M^2 d, \qquad C_{\text{iter}} \approx 2\, N^2 M, \tag{13}$$

which at the maximum of $M = 1024$ traps, $L = 2$, $d = 64$ and $N = 256$ give about 2.7×10^8 and 1.3×10^8 multiply-adds, so by operation count the network pass at the largest array is of the order of two iterations of the method it is meant to shorten. The start hologram of Eq. (5) costs one back-projection, as one iteration does. Training is offline and local, on one GPU, never in the continuous integration of the product.

The method is implemented in the engine of the product as the module `methods/learned/b3_warm_start.py` (the method), `learned/nets.py` (the network), `learned/torch_forward.py` (the differentiable iteration), `learned/training.py` (Algorithm 2) and `learned/heldout.py` and `learned/evaluation.py` (the held-out sets and the evaluation of Section 6). With no released network, the module answers with the iteration from

Parameter	Value	Origin
width d, layers, heads, feed-forward	64, 2, 4, 128	choice of the product, sized to stay under 1 MB
Fourier octaves ν	1, 2, 4, 8, 16, 32	choice of the product after [29]
maximum number of traps	1024	contract of the model slot
unroll K, freeze index in training	10, 3	choice of the product (Section 4.5)
loss constants 0.01, ω_0	0.01, 0.3	choice of the product
steps, batch, peak learning rate	6000, 8, 8×10^{-4}	choice of the product (AdamW [30], decay 10^{-5})
data seed, initialization seed	20261101, 7	recorded in the training configuration
freeze index of the engine default n_{fix}	12	Kim et al. [6]
iteration budget $K_{\max}$	60	evaluation protocol (Section 6)

Table 2: Parameters of the learned warm start and of the iteration it feeds, with their origin; the network and training values are those of the trained candidate whose configuration is recorded with its checkpoint.

its own random start and marks the result as a baseline, so that a missing model is never mistaken for a learned one.

5 Theory: what a start can and cannot change

The propositions of this section are exact statements about the iteration of Section 4.3 and the network of Section 4.4; the relation of Section 5.4 is a conjecture with the evidence for it. Together they say where a warm start can act and why the gain it can give is bounded by the freeze index.

5.1 Invariances

Proposition 1 (global phase)**.** *Let $c \in \mathbb{R}$ and let the start phases be shifted, $\theta'_m = \theta_m + c$ for all m. Then for every $k \geq 0$ the holograms satisfy $\phi'^{(k)} = \phi^{(k)} + c$ and the fields $V'^{(k)} = e^{ic} V^{(k)}$, so σ_k, η_k and u_k are the same sequences.*

Proof. For $k = 0$, Eq. (5) gives $\phi'^{(0)} = \arg\left(e^{ic} \sum_m a_m e^{i(\Delta^m + \theta_m)}\right) = \phi^{(0)} + c$, and Eq. (1) gives $V'^{(0)}_m = N^{-2} \sum_j A_j e^{i(\phi_j^{(0)} + c - \Delta_j^m)} = e^{ic} V_m^{(0)}$. Assume the claim for $k-1$. The relative amplitudes $B_{k-1}(m) = |V_m^{(k-1)}|/a_m$ do not change, so the weights of Eq. (6) do not change; the phases of Eq. (7) become $\psi'_{k,m} = \psi_{k,m} + c$ (and the frozen phase $\psi'_{n_{\mathrm{fix}}} = \psi_{n_{\mathrm{fix}}} + c$). Eq. (8) then gives $\phi'^{(k)} = \phi^{(k)} + c$, and Eq. (1) gives $V'^{(k)} = e^{ic} V^{(k)}$. The intensities are unchanged. □

Proposition 1 is why the loss of Eq. (12), which reads intensities only, needs no special treatment of the global phase, and why a regression onto converged phases would need one.

Proposition 2 (permutation equivariance)**.** *Let π be a permutation of the valid traps and $\pi\mathcal{T}$ the permuted trap set. The network of Eqs.* (9)–(11) *satisfies $g_\vartheta(\pi\mathcal{T}) = \pi\, g_\vartheta(\mathcal{T})$, and the start hologram of Eq.* (5) *is the same for $\mathcal{T}$ and $\pi\mathcal{T}$.*

Proof. The features of Eq. (9) and the embedding act on each trap separately, so they commute with π. In Eq. (10) the queries are permuted with the traps; the logits of query m over the keys are permuted with the keys, and the softmax and the weighted sum of the values are invariant to a permutation of the keys, so the output for trap m is unchanged; the additive mask follows its trap. The mean $\bar{\mathbf{h}}$ over the valid traps is invariant to π, and the readout of Eq. (11) acts row by row with $\bar{\mathbf{h}}$ and ϱ_m^2 attached to trap m. Hence θ_m follows trap m. The sum of Eq. (5) runs over all traps and does not depend on their order. □

The network has no positional index of any kind, so the order in which a user lists the traps cannot change the start, and the padded entries (removed by the mask) cannot change it either. The proposition is also the reason for a set network and against a recurrent or convolutional one: an ordering of traps would be an arbitrary input that the network would have to learn to ignore.

5.2 What the start changes

Proposition 3 (the start enters through the frozen phase)**.** *(a) For $n_{\mathrm{fix}} = 1$, every hologram $\phi^{(k)}$, $k \geq 1$, is a function of the trap set and of the vector $\psi_1 = \arg V^{(0)}$ alone. (b) For general n_{fix}, every $\phi^{(k)}$ with $k \geq n_{\mathrm{fix}}$ is a function of the trap set and of the trajectory $V^{(0)}, \ldots, V^{(n_{\mathrm{fix}}-1)}$, that is, of the start through $n_{\mathrm{fix}} - 1$ applications of Eqs.* (6)–(8)*.*

Proof. (a) With $n_{\mathrm{fix}} = 1$ Eq. (7) gives $\psi_k = \psi_1$ for all $k \geq 1$ and $g_1 = \mathbf{1}$, so $\phi^{(1)} = \arg \sum_m a_m e^{i(\psi_1 + \Delta^m)}$ depends on ψ_1 only. Then $V^{(1)}$ is a function of $\phi^{(1)}$, g_2 of $V^{(1)}$ by Eq. (6), and $\phi^{(2)} = \arg \sum_m g_{2,m} a_m e^{i(\psi_1 + \Delta^m)}$ of g_2 and ψ_1; the induction continues. (b) The frozen phase is $\psi_{n_{\mathrm{fix}}} = \arg V^{(n_{\mathrm{fix}}-1)}$ by Eq. (7), and the weights g_k for $k \leq n_{\mathrm{fix}}$ are functions of $V^{(1)}, \ldots, V^{(n_{\mathrm{fix}}-1)}$ by Eq. (6); after the freeze the sequence is generated by the same two rules from these quantities, so the start enters only through the trajectory up to $V^{(n_{\mathrm{fix}}-1)}$. □

Part (a) turns the warm-start problem at $n_{\mathrm{fix}} = 1$ into the prediction of M phases and nothing else: the network's θ_m become, to the extent that the cross terms of Eq. (5) are small, the frozen phases of the whole run. Part (b) holds for any deterministic iteration and constrains nothing by itself; its content here is that the start enters only through $V^{(0)}$ while the weights restart at one, so at the engine default $n_{\mathrm{fix}} = 12$ the start has to survive eleven applications of the iteration to matter. Whether it does is a property of the dynamics of the iteration, not of the network, and it is measured in Section 8: from the phases of a converged run (the best start one could hope to predict) the iteration at $n_{\mathrm{fix}} = 12$ still needs 13 iterations (median over ten geometries) against 14 from a random start.

5.3 An accounting bound on the weights

The weights restart at one whatever the start is, so a warm start does not transfer the correction that a converged run has stored in g. How long the weights need to rebuild it is bounded below by an accounting identity. Let $S_k = \max_m \log g_{k,m} - \min_m \log g_{k,m}$ be the log-spread of the weights after iteration k, and $s_k = \log\big(\max_m B_{k-1}(m) / \min_m B_{k-1}(m)\big)$ the allowance of the update at iteration $k \geq 2$.

Proposition 4 (allowance)**.** *For every run and every $k \geq 2$,*

$$S_k \;\leq\; \sum_{j=2}^{k} s_j. \tag{14}$$

Proof. By Eq. (6), $\log g_{k,m} = \log g_{k-1,m} + \log\langle B_{k-1}\rangle - \log B_{k-1}(m)$. The term $\log\langle B_{k-1}\rangle$ is common to all traps and cancels in the spread. The spread of a sum is at most the sum of the spreads, and the spread of $-\log B_{k-1}(m)$ over m is s_k. Hence $S_k \leq S_{k-1} + s_k$, and $S_1 = 0$ since $g_1 = \mathbf{1}$. □

If a converged run holds a weight spread $S^\star$, no run that starts from $g = \mathbf{1}$ reaches that spread before the iteration $k_{\mathrm{floor}} = \min\{k : \sum_{j=2}^{k} s_j \geq S^\star\}$. The bound is a statement about the restart; its tightness is measured, not assumed: we compute S_k and the allowance along the runs of Section 8 and compare k_{floor} with the iterations to 1 percent (Table 8).

5.4 A relation between the freeze index and the iterations

Conjecture 1 (iterations follow the freeze)**.** *For the random start and the network start on dense arrays (at least 40 traps) for which the iteration reaches 1 percent, the number of iterations to 1 percent is the freeze index plus a short tail, $k_1 = n_{\mathrm{fix}} + t$ with t of the order of one to four iterations on most geometries, and the start changes t and the probability of reaching 1 percent. The relation is a description of the starts tried here and has counterexamples, listed below.*

We do not prove the conjecture; the evidence is of four kinds, all in Section 8. (i) Before the freeze the spread of the default run stays at several percent: the median trajectory of the random start at $n_{\mathrm{fix}} = 12$ is 2.85 percent at iteration 11 and 2.61 percent at 12, and drops to 1.2 percent at 13, the first iteration that uses the frozen phase. (ii) The sweep of the freeze index puts the median k_1 of the random start at 2, 3, 4, 4 and 6.5 iterations above the freeze index for $n_{\mathrm{fix}} = 12, 8, 6, 4, 3$; these medians include the runs that reach the cap of the budget (41.1 percent of the runs at $n_{\mathrm{fix}} = 3$ and 28.9 percent at $n_{\mathrm{fix}} = 4$), and the excess of

the runs that converge alone was not computed, because the sweep stores medians and not the count of each run, so the tail of one to four is exceeded at $n_{\text{fix}} = 3$ and is not verified on converged runs. (iii) The network start at $n_{\text{fix}} = 12$ has the same median as the random start, and (iv) the phases of a converged run lower the median at $n_{\text{fix}} = 12$ by 1 iteration only. The counterexamples are in Table 7: the random start reaches 1 percent before the freeze index at $n_{\text{fix}} = 12$ on one of the ten geometries (`gen/09`), the converged phases do so on three (`gen/00`, `gen/07` and `gen/09`), and at $n_{\text{fix}} = 3$ the converged phases lower the median from 11.5 to 5.5 iterations and converge on three geometries where the random start never does. These are ten geometries and one seed. The conditions under which the relation fails are also in the data: a freeze that is too early locks phases that are incompatible with a uniform solution, and then the run never reaches 1 percent; the share of such runs rises as the freeze index falls (Table 6). A warm start can therefore act on the second term, on the share of runs that converge, and on the final spread, and could lower the first term only by lowering the index at which freezing is safe.

5.5 Consequences for the experiment

The propositions fix what the experiments must separate. Because the iteration depends on the start only through $\psi_{n_{\text{fix}}}$ (Proposition 3), a network start and a change of n_{fix} are two different handles on the same quantity, and an experiment that changes both at once cannot attribute the effect: the candidate network was trained and first evaluated at $n_{\text{fix}} = 3$ while its baseline ran at the default $n_{\text{fix}} = 12$. The ablation of Section 8 therefore crosses the two factors, and the sweep of n_{fix} with and without the network tests Proposition 3(a) directly at $n_{\text{fix}} = 1$. A fair comparison of a learned start therefore needs the baseline at the same freeze index as the candidate, in addition to the baseline at the published default: the first isolates the contribution of the network, the second measures the practical gain over the method as published. The first evaluation of the candidate compared it only with the second, and the difference between the two comparisons is the result of this paper. Proposition 4 gives a check that costs nothing: the weights cannot be the binding constraint if k_{floor} is far below the measured iterations.

6 Experimental design

6.1 Hypotheses

Three hypotheses were fixed, with their thresholds, before the network was trained, and a fourth was added after the first evaluation to explain its outcome; we label it as such.

H1 (speed). On geometries held out from training, the median number of iterations to 1 percent, k_1 of Eq. (3), of phase-fixed WGS from the network start is at most half that from the random start: the ratio of the medians is at least 2.

H2 (accuracy). The final efficiency and the final uniformity of the run from the network start differ from those of the random start by at most 1 percent (relative difference of the medians).

H3 (size). The exported network is at most 1 MB as an ONNX graph. Its inference time in a browser (a threshold of 10 ms in the product's research record) belongs to the same acceptance test; it was not measured here because H1 failed first.

H4 (attribution; added after the first evaluation). If the network start shortens the iteration, the gain is not reproduced by a random start with the same freeze index n_{fix}; if instead a random start with the same n_{fix} reproduces it, the gain belongs to the freeze index. The prediction of the second reading, which Section 5 motivates, is that a network start at the default $n_{\text{fix}} = 12$ shows no gain, and that the gain of the candidate at $n_{\text{fix}} = 3$ is the gain of $n_{\text{fix}} = 3$.

H1 and H2 are the acceptance test of the model slot of the product: a network that fails either is not released. H3 is met by the candidate. H4 is exploratory; its tests are the ablation of Section 8, and it does not change the status of H1 and H2.

6.2 Data and provenance

The held-out file was written once, before any training run, from seeded generators, and is identified by its sha256 digest (checked by the measurement script against the digest recorded with the rejected candidate).

Group	Geometries	Traps	Grid	Origin
generated, demanding	60	49–1024	256^2, pad 2	fresh seed; at least 40 traps; the random start needs more than 3 iterations
benchmark G1 to G6	6	36–1024	256^2 and 512^2	benchmark geometry records of the product

Table 3: Composition of the held-out set of 66 trap geometries: the generated geometries are filtered to be demanding for the baseline, and the benchmark geometries are those of the product's benchmark; neither group is used in training.

Training reads other seeds and never this file. Table 3 describes it. Its 60 generated trap sets come from the same generators as the training data (lattices, hexagonal lattices, rings, random sets with a minimum spacing and three-dimensional stacks) with a fresh seed, and are filtered to be demanding: at least 40 traps and a random-start phase-fixed WGS run, on a selection seed that is not an evaluation seed, that needs more than 3 iterations to reach 1 percent. A sparse array reaches 1 percent at the first iteration from any start, so it cannot show a warm start, and the filter looks only at the baseline, never at a model. The remaining 6 geometries are the six point-trap geometries of the benchmark of the product (G1 to G6: a 10×10 square lattice, a ring of 36 traps, 100 random traps, a quasi-periodic array of 119 traps, a three-dimensional lattice of $5 \times 5 \times 3$ traps and a 32×32 lattice of 1024 traps), on the grids of the benchmark, the last of which is 512^2 pixels. The quasi-periodic benchmark geometry belongs to a family absent from training. The generated sets have between 49 and 1024 traps (median 275). The engine setup of the generated sets is an SLM of 256×256 pixels of pitch 12.5 μm with a zero padding of 2, a lens of 250 mm and a wavelength of 850 nm.

6.3 Methods compared

Every method is run by the engine of the product on the same geometries, seeds and budget; no number is taken from a publication. The published method is phase-fixed WGS of Kim et al. [6] at its default freeze index $n_{\mathrm{fix}} = 12$ from the random superposition start of [2]. Two published iterations that precede it serve as references: the Gerchberg–Saxton iteration [1] and weighted GS [2], both from the random start. The proposed method is phase-fixed WGS from the network start. The ablation adds the arms that separate the two factors that the candidate changed at once, the start and the freeze index:

- random start, $n_{\mathrm{fix}} = 12$ (the published default);
- random start, $n_{\mathrm{fix}} = 3$ (the early freeze alone, no network);
- network start, $n_{\mathrm{fix}} = 12$ (the network alone, the published freeze);
- network start, $n_{\mathrm{fix}} = 3$ (the candidate as it was trained and first evaluated).

A sweep of n_{fix} over $1, 2, 3, 4, 6, 8$ and 12 for both starts (on every second generated geometry, 30 of them) and an oracle start (Section 6.5) complete the design. The network is the trained candidate: the checkpoint at the last of its 6000 steps, read from the training directory of the data root; nothing is retrained.

6.4 Metrics and statistics

The per-run quantities are those of Eqs. (2) and (3), read after each iteration by the engine. For a geometry g and an arm the per-geometry value is the median over the 3 seeds,

$$\tilde{k}_g = \mathrm{median}_{s \in \{301,302,303\}}\ k_1^{(g,s)}, \tag{15}$$

and the summary of an arm is the median over geometries, $\tilde{k} = \mathrm{median}_g\, \tilde{k}_g$; the ratio of two arms is the ratio of their summaries, $\rho = \tilde{k}_{\mathrm{ref}}/\tilde{k}_{\mathrm{arm}}$. The same seed gives the same random phases in every arm, so the arms are paired by geometry and seed. The uncertainty of $\tilde{k}$ and of ρ is a percentile bootstrap over geometries,

$$\mathrm{CI}_{95} = \big[\, q_{2.5}\{\tilde{k}^{*b}\}_{b=1}^{B},\ q_{97.5}\{\tilde{k}^{*b}\}_{b=1}^{B} \,\big], \qquad B = 10^4, \tag{16}$$

where $\tilde{k}^{*b}$ is the summary of the b-th resample of the 66 geometries with replacement [31], with a fixed seed for the resampling. Because k_1 is an integer capped at $60 + 1$ and the arms differ in how many runs hit the cap, we also report the share of runs that never reach 1 percent, the quartiles of $\tilde{k}_g$ and the median final σ.

The difference between two arms on the geometries is tested with the exact two-sided sign test on the paired $\tilde{k}_g$, ties dropped,

$$p = \min\Big\{1,\ 2\sum_{j=0}^{\min(n_+,n_-)} \binom{n_+ + n_-}{j} 2^{-(n_+ + n_-)}\Big\}, \tag{17}$$

with n_+ and n_- the numbers of geometries where the first arm needs more and fewer iterations. The sign test uses no distributional assumption and is robust to the heavy tail of the capped counts. Final-efficiency differences are the relative difference of the medians over geometries, as in the acceptance test.

6.5 Oracle start and weight accounting

Two further measurements probe the theory. The oracle start is the best start a predictor could hope for: the phases $\arg V_m$ of the final field of a random-start phase-fixed WGS run of 60 iterations on the same geometry and seed are used as the trap phases θ_m of Eq. (5), and the iteration is run again from there with $n_{\mathrm{fix}} = 12$, 3 and 1. It is computed on the first ten generated geometries with the first seed. The weight accounting runs the iteration with a wrapper that records S_k and s_k of Section 5.3 at every iteration, from the random start and from the oracle start, for $n_{\mathrm{fix}} = 12$ and $n_{\mathrm{fix}} = 3$, on the same ten geometries and seed, and checks Eq. (14) on every run.

The two probes answer different questions. The oracle start bounds what any predictor of trap phases could gain at a given freeze index, and its comparison with the network start tells whether the network is far from that bound or the bound itself is small. The accounting tests a specific explanation of the bound, that the restart of the weights limits the iteration, by comparing the lower bound it implies with the iterations that the run actually needs. Both are exploratory and are reported with their small samples.

6.6 Hardware and run times

The engine is the NumPy implementation of the product (the point-trap form for every geometry); the network runs in PyTorch on the CPU for the inference of each geometry, and the measurement of all arms was run with 12 worker processes on the development workstation (Intel Core i9-14900HX, 48 GB of RAM), in 37.6 minutes of wall-clock time. The network was trained on a single RTX 4070 Laptop GPU in 30.8 minutes. The generated trap sets and the held-out file do not depend on the machine; the wall-clock numbers do, and none of the conclusions rests on a time. The number of worker processes does not change any result, because each run is a pure function of its geometry, seed, start and freeze index, and the results are gathered by identifier. The measurement script checks the held-out digest before it starts and stops if it differs. Every random choice goes through the portable seeded generator of the engine, so a rerun reproduces the same numbers up to the floating point of the machine's fast Fourier transform.

7 Results

7.1 Training behaviour

The network trained for 6000 steps. The loss of Eq. (12) fell from 2.54 at the first logged step to an average of 0.97 over the last ten logged steps (Figure 2a), a decrease that the batch-to-batch variation of the trap sets makes noisy: the batches differ in the number of traps from one step to the next. The logged quantity that matters for the question is the spread after the last unrolled iteration, which the training script measures for the network phases and, on the same batch, for random phases (Figure 2b). Over the last twenty logged steps the median spread after ten iterations is 0.176 percent for the network phases and 0.143 percent for random phases at the training freeze index $n_{\mathrm{fix}} = 3$: the network is not better than random phases at the end of the unrolled iteration on the training distribution, although it is better at the first iteration (9.2 against 12 percent after iteration 1). The advantage the loss rewards at the start is gone by iteration ten. On the held-out geometries the advantage after the first iteration is absent as well: the median spread after iteration 1 is 31.11 percent for the network start and 30.35 percent for the random start (Table 4 gives the other quantities).

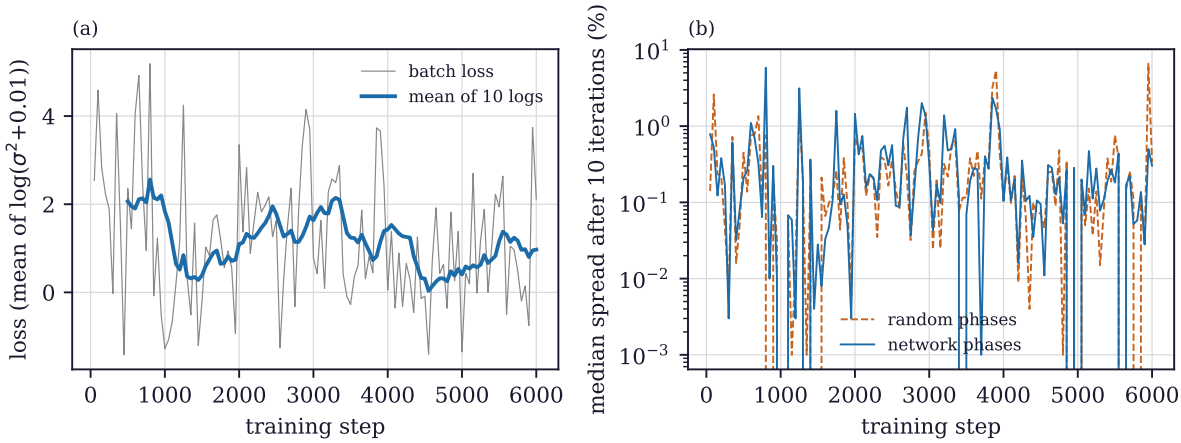


Figure 2: Training of the warm-start network: (a) loss of Eq. (12) per logged step with a mean of ten logs; (b) median spread after the last unrolled iteration for the network phases and for random phases on the same training batches, per logged step, on a logarithmic axis.

Start	n_{fix}	Median it. [95% CI]	Q1–Q3	Never (%)	Final σ (%)	Efficiency
random (engine default)	12	14 [14, 14]	13–15	6.1	0.00013	0.790
random	3	7.5 [6, 10.5]	5–61	32.8	0.217	0.757
network (candidate)	3	8 [7, 17]	5–61	34.8	0.216	0.756
network	12	14 [14, 14.5]	13–15	4.5	0.000222	0.791
Gerchberg–Saxton, random start	–	61 [61, 61]	–	100	16.3	0.821
weighted GS, random start	–	30 [27, 36]	–	6.6	0.472	0.808

Table 4: Iterations to a relative standard deviation of 1% on the 66 held-out geometries: median over geometries with a bootstrap 95% interval, quartiles, the share of runs that never reach 1% within 60 iterations, the median final spread σ in percent and the median final efficiency, for the four arms of phase-fixed WGS and two published references from the random start.

7.2 Main comparison on held-out geometries

Table 4 gives the four phase-fixed WGS arms and the two published references on the 66 held-out geometries (3 seeds each, a budget of 60 iterations). The published default (random start, $n_{\text{fix}} = 12$) reaches 1 percent at a median of 14 iterations, with a tight spread (quartiles 13 to 15) and 6.1 percent of runs that do not reach it within the budget; the references are slower or less reliable: the unweighted Gerchberg–Saxton iteration does not reach 1 percent within the budget on 100 percent of the runs, and weighted GS reaches it at a median of 30 iterations (interval 27 to 36), failing on 6.6 percent of the runs. The candidate, the network start at $n_{\text{fix}} = 3$, reaches 1 percent at a median of 8 iterations, with a 95% bootstrap interval of 7 to 17, and the ratio of the default's median to the candidate's is 1.75 (interval 0.85 to 2). The point estimate is below the required factor of two and the interval includes values well below it; it also includes one, so the speed-up of the candidate over the default is not established at the 95% level by this interval. The paired sign test over geometries gives the same reading: the candidate needs fewer iterations than the default on 37 geometries and more on 23 (6 ties), $p = 0.092$.

The final quantities show the price of the early freeze, whatever the start. The final median spread is 0.00013 percent at $n_{\text{fix}} = 12$ and 0.217 (random) and 0.216 (network) percent at $n_{\text{fix}} = 3$, and the final efficiency is 0.790 at the default and 0.757 and 0.756 with the early freeze. The relative difference of the medians of the final efficiency between the candidate and the default is 4.3 percent, between a random start with the early freeze and the default 4.2 percent, and between the candidate and the random start with the same freeze 0.1 percent; the final uniformity median differs by 0.7 percent in the committed record of the candidate. The candidate reproduces the committed record of its first evaluation (median 8 against 14, factor 1.75, efficiency difference 4.2 percent): the arms of this study differ from that evaluation by the network being read from its training checkpoint rather than from the exported graph.

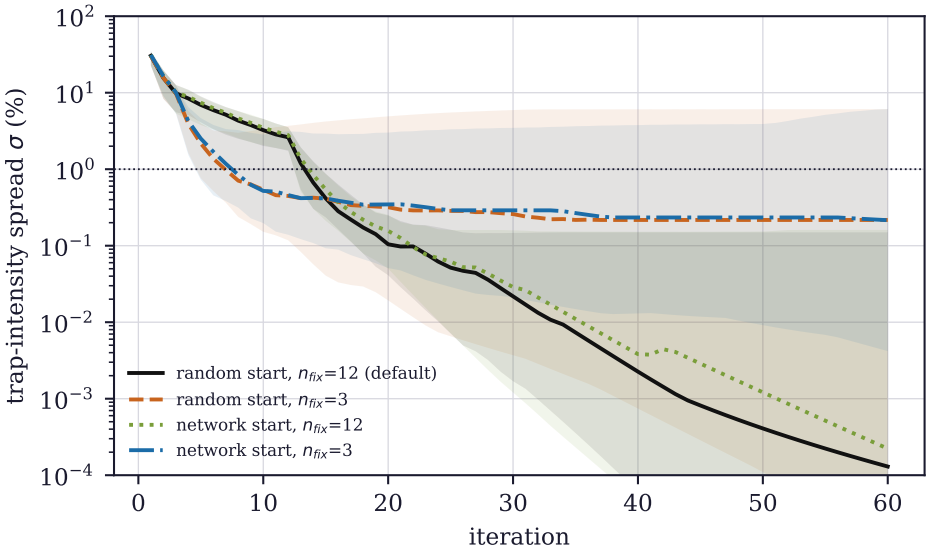


Figure 3: Median relative standard deviation of the trap intensities along the 60 iterations of phase-fixed WGS on the 66 held-out geometries, for the four arms (random or network start, $n_{\text{fix}} = 3$ or 12); the bands are the interquartile range across geometries and the dotted line is the 1% threshold.

7.3 Outcome of the hypotheses

H1 is not supported: the factor is 1.75, below 2, and the interval 0.85 to 2 does not exclude no gain at all. H2 is not supported: the final efficiency of the candidate is 4.3 percent from the default's, above the 1 percent allowed. H3 is met: the exported graph is 338 987 bytes. H4 is supported by the three arms of Section 8: a random start with the same early freeze gives a factor of 1.87 (interval 1.38 to 2.33, sign test $p = 0.0032$) and the same efficiency loss, and the network at the default freeze index gives a factor of 1. Because H1 failed, the model is not released, and the slot of the product answers with the random start of the iteration.

Taken together, the acceptance test is failed on its first two clauses and passed on the third, which is the pattern of a start that is small and ineffective, not of a start that is large and inaccurate. The effect that remains with respect to the published default, the lower median iterations, is reproduced by the control without a network; the effect of the network beyond the control is not distinguishable from zero in any of the measured quantities, the median, the share of runs that converge, the final spread and the final efficiency.

7.4 Convergence along the iterations

Figure 3 shows the median spread of each arm along the iterations, with the interquartile band across geometries. The curves of the two arms with the same freeze index are almost on top of each other: the two $n_{\text{fix}} = 12$ arms follow the staircase of the default, with the spread at several percent until the freeze (2.85 percent at iteration 11 for the random start, 3.09 for the network) and a drop at the first iteration after it (1.2 and 1.37 percent at iteration 13); the two $n_{\text{fix}} = 3$ arms drop at iteration 4 (3.92 percent for the random start, 4.25 for the network) and then flatten at a spread of about 0.4 percent. The plateau is why the $n_{\text{fix}} = 3$ arms do not reach 1 percent on a third of the runs: for the median geometry the frozen phases allow 1 percent, for the geometries in the upper quartile they do not. Figure 4 shows the distribution of the iterations over the geometries: the early-freeze arms have their first quartile at 5 iterations and their third quartile at the cap, while the default has quartiles of 13 and 15.

7.5 Geometry by geometry

Figure 5 compares the arms geometry by geometry. Against a random start with the same freeze index (Figure 5a) the points lie on the diagonal: the network start needs the same number of iterations on 47 of the 66 geometries, fewer on 6 and more on 13 (sign test $p = 0.17$), so if anything the network is slightly slower than the random start at $n_{\text{fix}} = 3$, a difference that is not distinguishable from none. Against the default (Figure 5b) the network start needs fewer iterations on 37 geometries and more on 23, the latter being mostly

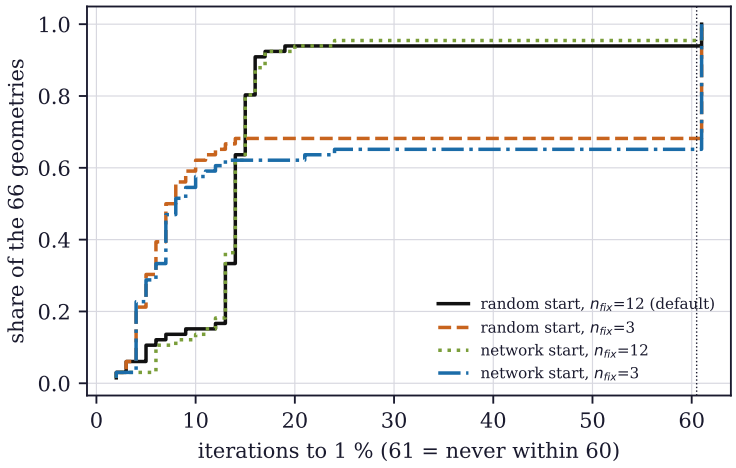


Figure 4: Empirical distribution of the iterations to 1% over the 66 held-out geometries for the four arms; the value $60 + 1$ marks the geometries that never reach 1% within the budget, which is where the early-freeze arms leave about a third of the geometries.

Arm	Network start	Early freeze	Median it.	Ratio to the default [95% CI]	Sign test p
engine default	no	no	14	1	–
early freeze only	no	yes	7.5	1.87 [1.38, 2.33]	0.0032
network only	yes	no	14	1.00 [1.00, 1.00]	0.29
network and early freeze	yes	yes	8	1.75 [0.85, 2.00]	0.092

Table 5: Ablation of the learned warm start on the 66 held-out geometries: the network start and the early freeze index ($n_{\text{fix}} = 3$ against the default 12) crossed, with the median iterations to 1%, the ratio of the default's median to each arm's with a bootstrap 95% interval, and the exact two-sided sign test against the default.

geometries on which it does not converge at all. Of the 66 geometries, 20 reach the cap with both early-freeze arms, 1 only with the random start and 3 only with the network start. The medians separated by group are 14 (random, $n_{\text{fix}} = 12$), 8 (random, $n_{\text{fix}} = 3$) and 8.5 (network, $n_{\text{fix}} = 3$) on the generated geometries, and 13.5, 6 and 5 on the six benchmark geometries.

8 Ablation

8.1 Network start against early freeze

The candidate changed two things at once with respect to the published default: the start phases and the freeze index. Table 5 separates them on the same 66 geometries and seeds, and Figure 6 shows the medians with their intervals and the share of runs that never reach 1 percent. The early freeze alone, with no network, lowers the median from 14 to 7.5 iterations, a factor of 1.87 (interval 1.38 to 2.33), and the paired sign test over geometries rejects equality (40 geometries need more iterations with the default, 17 fewer, 9 ties, $p = 0.0032$). The network alone, with the published freeze, does not change the median (14 against 14, factor 1; 8 against 14 geometries, $p = 0.29$). The combination of the two, the candidate, gives 8 iterations, a factor of 1.75, which is below the factor of the early freeze alone (1.87), and its sign test against the default has $p = 0.092$, larger than that of the freeze alone. In every comparison the arm with the network is not better than the arm without it that has the same freeze index.

The efficiency tells the same story. The final efficiency is 0.790 at the default, 0.791 for the network at $n_{\text{fix}} = 12$ and 0.757 and 0.756 with the early freeze: the loss of efficiency that failed H2 is caused by the freeze (4.2 percent for a random start with it) and the network adds 0.1 percent to it. The share of runs that never reach 1 percent shows what the early freeze buys the median with: 32.8 percent for the random start at $n_{\text{fix}} = 3$ and 34.8 percent for the network, against 6.1 and 4.5 percent at the default. A statistic that does not look at the tail, the median, hides that the early freeze fails to converge on about a third of the runs.

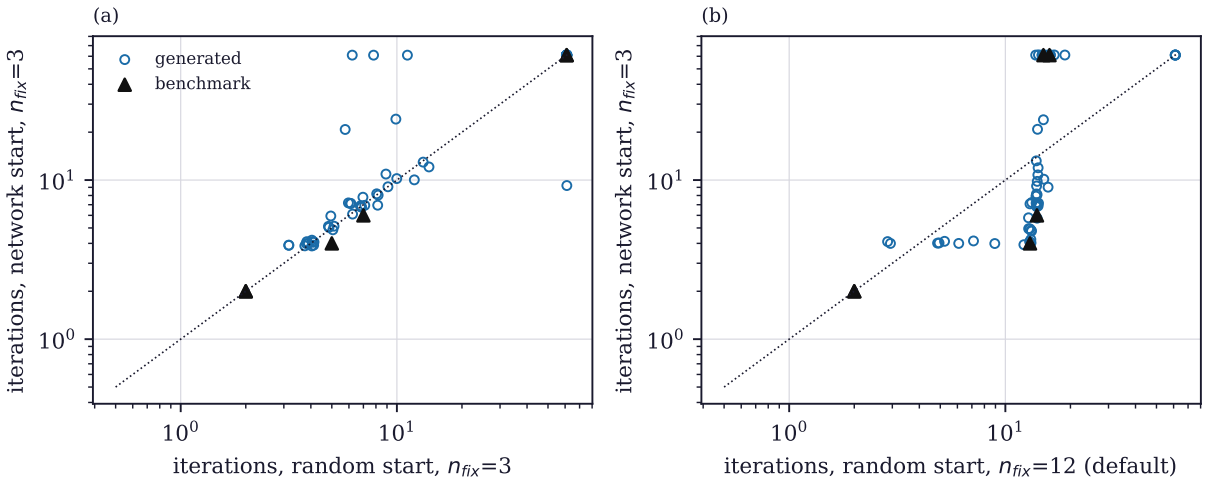


Figure 5: Iterations to 1% per held-out geometry, network start at $n_{\text{fix}} = 3$ (vertical axis) against (a) the random start at $n_{\text{fix}} = 3$ and (b) the random start at the default $n_{\text{fix}} = 12$ (horizontal axis), logarithmic axes, with a small jitter for ties; circles are generated geometries, triangles the six benchmark geometries, the dotted line is equality and $60 + 1$ marks a run that never reaches 1%.

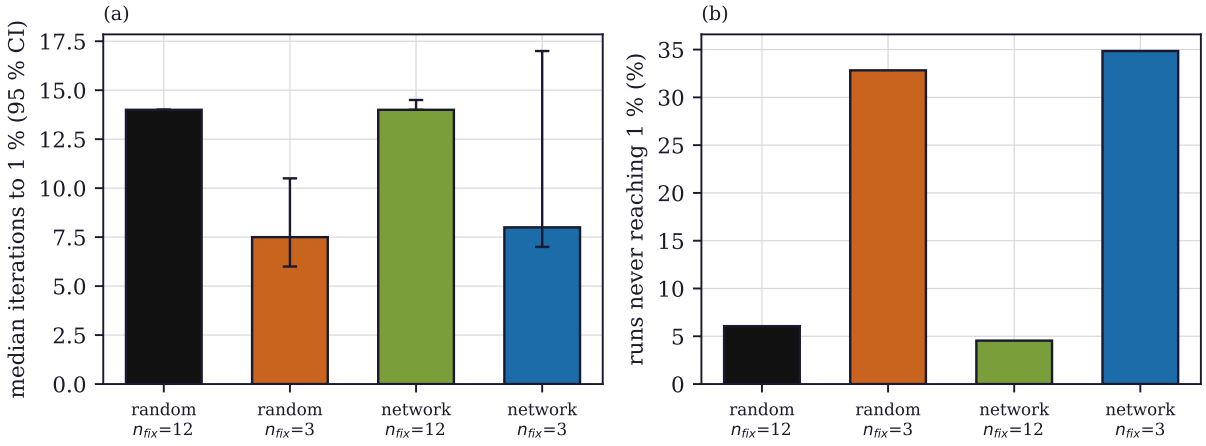


Figure 6: Ablation on the 66 held-out geometries: (a) median iterations to 1% with the bootstrap 95% interval for the four arms; (b) share of the runs (66 geometries, 3 seeds) that do not reach 1% within 60 iterations, which is the price of the early freeze whatever the start.

8.2 Sweep of the freeze index

Table 6 and Figure 7 give the median iterations, the share that never converge and the final spread for the freeze index from 1 to 12, for both starts, on the 30 geometries of the sweep. The median for the random start falls from 14 iterations at $n_{\text{fix}} = 12$ to 11 at 8, 10 at 6, 8 at 4 and 9.5 at 3, while the share of runs that never reach 1 percent rises from 7.8 to 12.2, 14.4, 28.9 and 41.1 percent. At $n_{\text{fix}} = 2$ the median is the cap (the run does not converge on most geometries) and at $n_{\text{fix}} = 1$ it is too: 76.7 percent of the runs never converge. The final spread rises as the freeze index falls, from 0.00013 percent at 12 to 0.512 at 3. The relation is a trade between iterations and the probability of reaching 1 percent, with a minimum of the median near $n_{\text{fix}} = 4$.

The network start follows the random start along the whole sweep. At $n_{\text{fix}} = 1$, where Proposition 3(a) says the run is a function of the start phases alone and the network would have its fullest effect, the network start reaches 1 percent in the median in 61 iterations and fails to converge in 80 percent of the runs, against 76.7 percent for the random start. At $n_{\text{fix}} = 2$ the shares are 70 and 61.1 percent, at $n_{\text{fix}} = 3$ 40 and 41.1, at $n_{\text{fix}} = 4$ 23.3 and 28.9, at $n_{\text{fix}} = 6$ 20 and 14.4 and at $n_{\text{fix}} = 12$ 6.7 and 7.8. The medians differ by at most one iteration wherever both are below the cap. The final spread of the network is lower than the random start's at several indices on this subset (at $n_{\text{fix}} = 12$, 1.91e-05 against 0.00013 percent), but it is higher over

n_{fix}	Random: median it.	Never (%)	Final σ (%)	Network: median it.	Never (%)	Final σ (%)
1	61	76.7	34.3	61	80	52.2
2	61	61.1	2.32	61	70	20.1
3	9.5	41.1	0.512	10.5	40	0.213
4	8	28.9	0.202	9	23.3	0.18
6	10	14.4	0.0742	10	20	0.0371
8	11	12.2	0.00997	12	16.7	0.0352
12	14	7.8	0.00013	14	6.7	1.91e-05

Table 6: Sweep of the freeze index n_{fix} on 30 of the held-out geometries (every second generated geometry, 3 seeds): median iterations to 1% (the value 61 means the median run does not reach 1% within 60 iterations), share of runs that never reach it and median final spread, for a random start and for the network start.

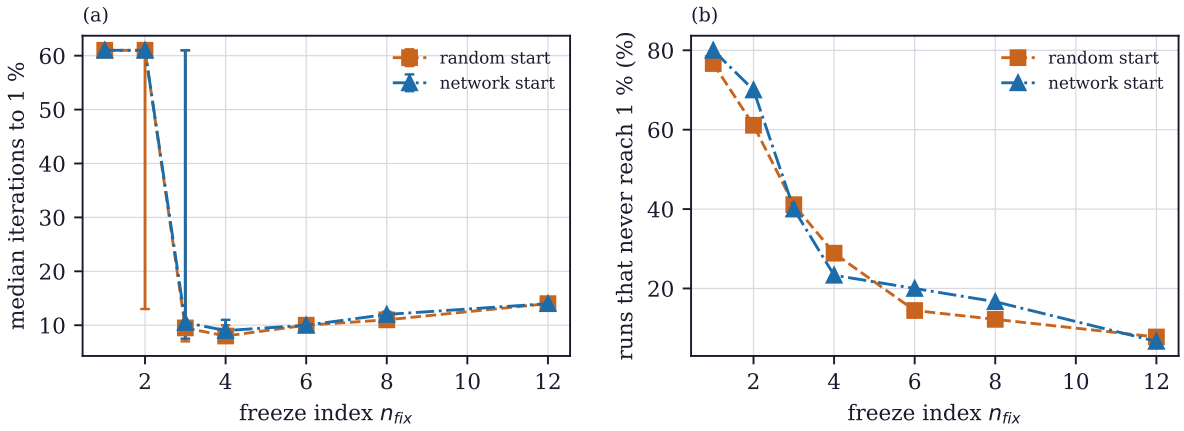


Figure 7: Sweep of the freeze index n_{fix} on 30 held-out geometries for the random and the network start: (a) median iterations to 1% with the bootstrap 95% interval; (b) share of runs that never reach 1% within 60 iterations. The two starts follow each other at every index.

the full 66 geometries (0.000222 against 0.00013 percent), so the direction of the final-spread difference is not consistent and we do not claim one.

8.3 Oracle start

The strongest start a predictor could be asked for is the converged solution of the iteration itself. Table 7 and Figure 8 give, for ten generated geometries and the first seed, the iterations to 1 percent from the phases of a converged random-start run. With the default freeze, the oracle start needs a median of 13 iterations against 14 from the random start, a saving of 1 iteration; at $n_{\text{fix}} = 3$ it needs 5.5 against 11.5, and with the freeze at the first iteration 5.5. On the geometry that does not converge for any arm (the largest array of the ten, `gen/06` in Table 7) the oracle does not help either. Two readings follow. First, at the published freeze index the start cannot be worth more than about one iteration, whoever predicts it: the network at $n_{\text{fix}} = 12$ has nothing to improve. Second, with an oracle the early freeze converges on geometries on which the random start with the same freeze never does (`gen/01`, `gen/03` and `gen/08`), which is the part of the gap a perfect predictor could close; the network, with its phases, does not close it (Table 6). The predicted phases are not those of a converged solution; whether the trap list determines such phases is not tested here, and the converged phases of a dense array depend, in these runs, on the random start that produced them.

8.4 Weight accounting

Table 8 and Figure 9 apply Proposition 4 to the runs of the oracle comparison. The bound of Eq. (14) holds on all 10 of 10 runs of each of the four conditions. It is loose: the weight spread of a converged run, a median of 0.71 for the random start at $n_{\text{fix}} = 12$, is within reach of the allowance after a median of 2 iterations (k_{floor}), while the median run needs 14 iterations to reach 1 percent. With an oracle start at $n_{\text{fix}} = 12$ the corresponding numbers are 4 and 13. The restart of the weights at one therefore does not explain why 1

Held-out geometry	Traps	Random, 12	Oracle, 12	Random, 3	Oracle, 3	Oracle, 1
gen/00	236	13	11	7	5	5
gen/01	345	18	14	61	8	8
gen/02	448	14	13	16	6	6
gen/03	456	15	13	61	7	6
gen/04	133	14	13	7	5	5
gen/05	102	13	13	5	5	4
gen/06	768	61	61	61	61	61
gen/07	679	13	8	4	4	4
gen/08	450	16	14	61	9	8
gen/09	140	5	2	4	2	2

Table 7: Iterations to 1% on ten held-out geometries (first seed) from a random start and from the oracle start (the phases of a converged random-start run) for freeze indices 12, 3 and 1; the value 61 means 1% is not reached within 60 iterations.

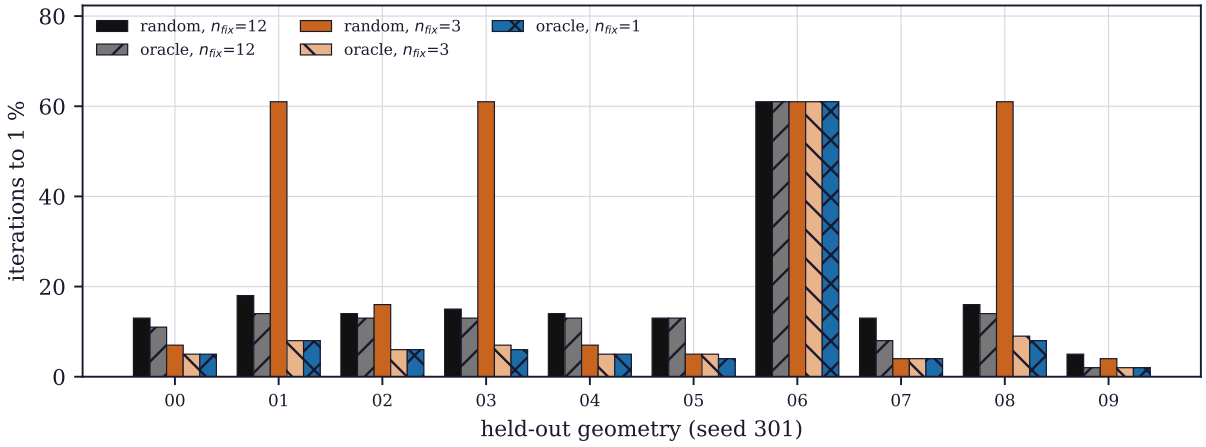


Figure 8: Iterations to 1% on ten held-out geometries from a random start and from the oracle start for freeze indices 12, 3 and 1 (first seed); the bars at the top of the axis are runs that do not reach 1% within the budget.

percent arrives at iteration 13 or later: the accounting identity permits the weights to rebuild their spread long before. This rules out the explanation that the weights, which restart, are the bottleneck, and it points to the freeze index as the quantity that sets the iteration count.

9 Discussion

9.1 Why the network does not help

The measurements give three reasons, in decreasing order of how directly they are measured. First, the iteration the network feeds forgets its start. At the published freeze index the run of the iteration passes through eleven applications of the update before the focal phase is frozen (Proposition 3), and the phases of a converged run, the best start that was constructed, save 1 iteration on ten geometries and one seed (Table 7), which is consistent with a small value of a phase predictor at $n_{\mathrm{fix}} = 12$ and is not a bound over all starts, and the network at $n_{\mathrm{fix}} = 12$ saves none (Table 5). Second, where the start does matter, at an early freeze, what matters is whether the frozen phases are compatible with a uniform solution, and whether the trap list determines such phases is not tested here: the oracle start, whose converged phases the trap list does not supply, converges on geometries where a random start does not and halves the median at $n_{\mathrm{fix}} = 3$ (11.5 to 5.5), while the network, trained on a loss that rewards low spread after ten iterations, does not (Table 6, Table 7). That headroom of a factor of two is the open question that this network leaves. Third, the training objective did not transfer to the held-out geometries: during training the network lowered

Run (seed 301, ten geometries)	Median final S	Median k_{floor}	Median it. to 1%	Bound holds
random start, $n_{\text{fix}} = 12$	0.71	2	14	10 of 10
random start, $n_{\text{fix}} = 3$	1.43	3	11.5	10 of 10
oracle start, $n_{\text{fix}} = 12$	0.67	4	13	10 of 10
oracle start, $n_{\text{fix}} = 3$	0.80	5.5	5.5	10 of 10

Table 8: Weight accounting on ten held-out geometries (first seed): median final log-spread S of the weights, the first iteration k_{floor} at which the cumulative allowance of Eq. (14) reaches that spread, the median iterations to 1%, and the number of runs on which the bound holds, for the random and the oracle start at two freeze indices.

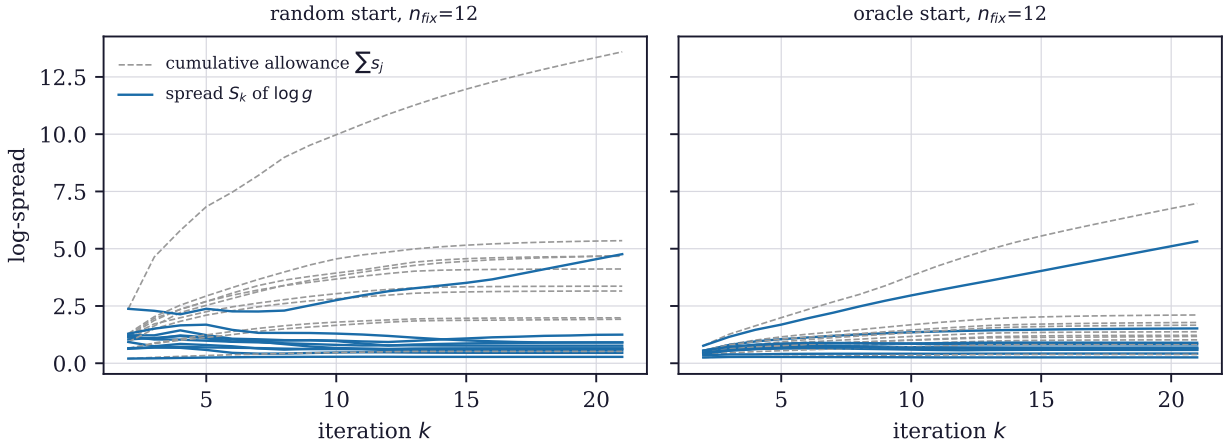


Figure 9: Log-spread S_k of the weights after iteration k (solid) and the cumulative allowance $\sum_{j \le k} s_j$ of Eq. (14) (dashed), for ten held-out geometries at $n_{\text{fix}} = 12$, from the random start (left) and the oracle start (right); the solid curves stay below the dashed ones on every run.

the spread after the first iteration from 12 to 9.2 percent on the training distribution, an advantage that is gone by iteration ten (0.176 against 0.143 percent) and absent after the first iteration on the held-out geometries (31.11 against 30.35 percent). We cannot say from these runs whether a longer or differently designed training would move the third reason; the first two do not depend on the training.

9.2 The gain belongs to the freeze index

The reduction in median iterations that the candidate shows with respect to the default is real and is reproduced without a network: a random start with the same early freeze needs 7.5 iterations. The sweep of Section 8.2 puts it in context: the median iterations fall steadily as the freeze index falls from 12 to 4 and the share of runs that never converge rises at the same time, from 7.8 to 28.9 percent. An early freeze is a bet that the phases at that moment are good enough, and on this model the bet fails on a share of the geometries that grows as the freeze comes earlier. A practitioner who wants fewer iterations on arrays of the type of the held-out set can therefore lower n_{fix} and accept the failure share; we did not test any remedy for the failed runs, such as a restart with a later freeze, and we make no recommendation. The default of Kim et al. [6] was chosen on their own array and camera; we did not test their array, and nothing in these results says that their choice was suboptimal for it. What the results say is that on the dense generated arrays of this study, in the scalar model, the default is a conservative point of a trade between iterations and reliability, and a learned start does not move the trade.

9.3 Cost of the network against what it can save

By operation count (Section 4.7) the network pass costs of the order of two iterations of the point form at the largest array of 1024 traps and less on smaller ones, because its attention is quadratic in M while an iteration is linear in it. A start that saved about one iteration at $n_{\text{fix}} = 12$ would not repay a pass of that size on the largest arrays, and for arrays of a hundred traps the pass is much cheaper than an iteration, so the net balance is a question of the array size and of the saving, which is zero here. We did not measure the

inference time of the exported graph in a browser, so the cost statement is an operation count, not a timing.

The balance is different for the many holograms of a sequence, where each hologram starts from its predecessor and no start has to be predicted, and for arrays smaller than a hundred traps, where the iteration is cheap and the median iterations to 1 percent are already few. The cost statement therefore holds for the dense, large arrays for which a warm start was proposed, which are also the arrays where the pass is the most expensive.

9.4 The explanation by the weights

The hypothesis that a warm start should be hard because the weights restart at one is not supported by the accounting of Section 8.4: the allowance permits the weights to rebuild their spread in 2 iterations, far before 1 percent arrives. The iteration count is set by the freeze, not by the rebuilding of the weights, in the data of this study. The hypothesis follows from the structure of the iteration and was tested by the measurement of Table 8, which did not support it. The proposition is a true inequality and stays; the hypothesis is dropped. An explanation that follows from the structure of an algorithm is a hypothesis, and the quantity it predicts (here a lower bound on the iterations) can be computed on every run at no cost; computing it shows that the bound is far below the observed counts. The same accounting applied to the oracle start gives the same answer, so the conclusion does not depend on the quality of the start. What the accounting leaves open is why the freeze index sets the count, for which the evidence is that of Conjecture 1, with its counterexamples, and no proof.

9.5 What the result does not close

A label-based warm start (regressing converged phases on a family of geometries where they are determined, for instance regular lattices) or a modified freeze rule that lets a start carry information through the iteration would be different designs, not tunings of this one, and the results here neither support nor exclude them. What the data suggest as a quantity to look for in a future design is the largest freeze index at which a given start still converges, and the share of geometries on which it does not. A start that moved that curve, one that converged on the geometries where a random start with an early freeze fails, would be a warm start in the sense of the proposal; the oracle start does it on three geometries of ten, and the network start does it on 1 of the 66 held-out geometries while failing on 3 where the random start with the same freeze converges. We also did not test whether a network trained at the published freeze index with a longer unroll would behave differently from the candidate; the single earlier trial at that setting reached about the random level, and it is the only evidence we have on the question. The oracle start suggests a saving of about one iteration for phase starts at the published freeze on these ten geometries, but it is one particular start (the converged phases of a single random-start run, one seed), so it is not a bound over all starts.

10 Limitations and threats to validity

10.1 Internal validity

The negative result concerns one trained network: one architecture (a two-block set transformer of width 64), one training run of 6000 steps with one initialization seed and one data seed, one loss (Eq. (12)) and one training freeze index ($n_{\mathrm{fix}} = 3$). We did not train other widths, depths, losses, unroll lengths or freeze indices for this study, so we cannot say that no network of the family would do better; a negative result on one candidate is a negative result on that candidate. The variation across the training seeds, which would give the uncertainty of the network's contribution itself, was not measured. The evaluation has 66 geometries and 3 seeds, and the integer-valued, capped statistic produces coarse medians: the intervals of the arms with the early freeze are wide (7 to 17 for the candidate) because the distribution of k_1 is bimodal, with a mass near the median and a mass at the cap. The sign test is the more informative comparison here, and we report both.

The ablation was designed after the first evaluation had failed (hypothesis H4), so its tests are exploratory; the sweep and the oracle start were added for the same reason. The sweep and the weight accounting use subsets (30 and ten geometries) that were chosen by position in the generated list, not at random, and the oracle start uses one seed. The numbers of the sweep, the oracle and the accounting carry no intervals beyond those shown.

10.2 External validity

The model is the scalar Fourier model with ideal pixels (Section 3.4), without pixel crosstalk, phase quantization, flicker or an input beam other than the Gaussian one; an instrument adds effects that change the trap intensities after the hologram is computed and that camera feedback absorbs, and the number of iterations of the phase-fixed iteration on a real device is not what we measure. The geometries are point traps without vortices, at most 1024 in number, on grids of 256^2 and 512^2 pixels; the held-out set is filtered to be demanding for the random start of the default iteration, so the conclusion holds for dense arrays that need more than three iterations and says nothing about sparse arrays, which any start solves in one. The training generators and the held-out generated geometries are the same families with different seeds, so the held-out set tests generalization over instances of the families and over the unseen benchmark families only in as far as the six benchmark geometries show; six geometries do not give a statistic.

10.3 Construct validity

The cost of a hologram is measured as iterations to 1 percent of the relative standard deviation, with a cap. Iterations are the unit of the acceptance test, but they are not time: the network pass has a cost (Section 9.3) that was counted but not timed, and iterations do not show that the early freeze lowers the final efficiency by about 4.2 percent. A threshold other than 1 percent would give other counts; we did not sweep it. The median over geometries of the median over seeds is the statistic of the acceptance test; we kept it for comparability and added the share of runs that do not reach the threshold, which the median hides.

A final construct concern is the choice of the baseline: the random start of the published default, run by the engine of the product with the portable generator, is the reference of the acceptance test. Another implementation of the same iteration, with a different random generator, would give different seeds but the same distribution of runs; the engine implementation of the iteration was compared with an independent implementation by the product, and we did not rerun that comparison here.

10.4 Scope of the claim

The claim that the study sustains is the following. On the held-out geometries of this study, in the scalar Fourier model, a network start trained as described does not reduce the iterations of phase-fixed WGS to 1 percent by the factor of two that the acceptance test required, and its gain at an early freeze is the gain of the early freeze without a network. The claim does not extend to other networks, other devices, hardware, other thresholds or a different iteration. It does not say that learned initializations cannot help other iterative hologram methods, which we did not test. The statement is tied to the held-out set, to the three seeds and to the budget of 60 iterations, and every number that sustains it is produced by the script of Section 11. The model is not released, and the product's slot for it answers with the random start of the iteration.

11 Reproducibility

The code, the committed records and the scripts that regenerate every number, table and figure of this paper are in the repository `https://github.com/fsantibanezleal/CEFOP_DinHot` (Apache-2.0), under `manuscripts/learned-warm-start/`. The studio that hosts the method runs at `https://dinhot.fasl-work.com`. The record of the rejected candidate is `data/derived/models/warmstart-model-rejected.json`; the method is the module `b3_warm_start.py`, and the held-out set, the training and the evaluation are in the package `learned/` of the engine.

Commands. From the repository root, with the GPU environment of the product (it holds PyTorch; only the CPU is used by the measurement):

- `python manuscripts/learned-warm-start/data/measure.py -workers 12` runs every arm, the sweep, the oracle start and the weight accounting, and writes `warm.json`, `warm_numbers.tex` and the table files; it took 37.6 minutes with 12 worker processes on the workstation of Section 6.6. With `-emit-only` it rewrites the macro and table files from `warm.json` without rerunning.
- `python manuscripts/learned-warm-start/figures/make_figs.py` regenerates the nine vector figures from `warm.json`.
- Three passes of `pdflatex` in `manuscripts/learned-warm-start/tex/` build the paper.

Data and artifacts. The held-out file and the checkpoint of the candidate are in the learned directory of the data root, named by the environment variable `DINHOT_LEARNED_DIR`; they are not in the repository. The held-out file is regenerated bit for bit by the function `build_warm` of `learned/heldout.py` from its recorded seeds, and its sha256 digest, which the measurement script checks against the committed record before it runs, is the identity check. The checkpoint comes from the training function of Algorithm 2 with the data seed 20261101 and the initialization seed 7; the exported graph is not committed because the model is not released. Every random choice goes through the portable seeded generator of the engine, so a rerun reproduces the numbers up to the floating point of the machine's fast Fourier transform. The numbers of the text are macros written by the measurement script, so none of them is typed.

12 Conclusions

We built a learned warm start for phase-fixed weighted Gerchberg–Saxton: a permutation-equivariant set network with 80 899 parameters that predicts one initial phase per trap, trained without labels through an unrolled copy of the iteration, with an acceptance test fixed in advance. On 66 held-out geometries it does not meet the test. The network start with an early phase freeze reaches a relative standard deviation of 1 percent in a median of 8 iterations (95% interval 7 to 17) against 14 for the random start of the published method, a factor of 1.75 (interval 0.85 to 2) where two was required, and its final efficiency is 4.3 percent from the published method's where 1 percent was allowed.

The four-arm ablation attributes the gain to the freeze index and not to the network: a random start with the same early freeze needs 7.5 iterations, the network at the published freeze index needs 14, and a start from the phases of a converged run saves only 1 iteration at that index (ten geometries, one seed). The early freeze has a price that the median hides: 32.8 percent of the runs never reach 1 percent within 60 iterations, against 6.1 percent for the published default, and the final efficiency is about 4.2 percent lower. The accounting bound on the weights holds on every run and is loose, so the restart of the weights does not explain the iteration count on these runs; the data are consistent with the freeze index setting it (Conjecture 1, which has counterexamples and is not established).

The model is not released. The questions that remain are different designs, not tunings: a start trained on labels where converged phases are determined, or a change of the freeze rule that lets a start carry information through the iteration. The results hold on a scalar Fourier model of the device and for one trained network; the experiment that would extend them is a measurement on an instrument with its device model and camera feedback, and a study of the variation across training seeds.

References

[1] R. W. Gerchberg and W. O. Saxton, "A practical algorithm for the determination of phase from image and diffraction plane pictures," *Optik* 35(2), 237–246, 1972 (no DOI; scan: `https://web.archive.org/web/20160328053000/http://www.u.arizona.edu/~ppoon/GerchbergandSaxton1972.pdf`).

[2] R. Di Leonardo, F. Ianni and G. Ruocco, "Computer generation of optimal holograms for optical trap arrays," *Opt. Express* 15(4), 1913, 2007. `https://doi.org/10.1364/OE.15.001913`.

[3] S. Bianchi and R. Di Leonardo, "Real-time optical micro-manipulation using optimized holograms generated on the GPU," *Comput. Phys. Commun.* 181(8), 1444–1448, 2010. `https://doi.org/10.1016/j.cpc.2010.04.012`.

[4] M. Polin, K. Ladavac, S.-H. Lee, Y. Roichman and D. G. Grier, "Optimized holographic optical traps," *Opt. Express* 13(15), 5831, 2005. `https://doi.org/10.1364/OPEX.13.005831`.

[5] G. Whyte and J. Courtial, "Experimental demonstration of holographic three-dimensional light shaping using a Gerchberg–Saxton algorithm," *New J. Phys.* 7, 117, 2005. `https://doi.org/10.1088/1367-2630/7/1/117`.

[6] D. Kim *et al.*, "Large-scale uniform optical focus array generation with a phase spatial light modulator," *Opt. Lett.* 44(12), 3178, 2019. `https://doi.org/10.1364/OL.44.003178`.

[7] H. Kim, M. Kim, W. Lee and J. Ahn, "Gerchberg–Saxton algorithm for fast and efficient atom rearrangement in optical tweezer traps," *Opt. Express* 27(3), 2184, 2019. `https://doi.org/10.1364/OE.27.002184`.

[8] W. Lee, H. Kim and J. Ahn, "Defect-free atomic array formation using the Hungarian matching algorithm," *Phys. Rev. A* 95(5), 053424, 2017. `https://doi.org/10.1103/PhysRevA.95.053424`.

[9] I. H. A. Knottnerus, Y. C. Tseng, A. Urech, R. J. C. Spreeuw and F. Schreck, "Parallel assembly of neutral atom arrays with an SLM using linear phase interpolation," *SciPost Phys.* 19(4), 118, 2025. https://doi.org/10.21468/SciPostPhys.19.4.118.

[10] F. Nogrette *et al.*, "Single-atom trapping in holographic 2D arrays of microtraps with arbitrary geometries," *Phys. Rev. X* 4(2), 021034, 2014. https://doi.org/10.1103/PhysRevX.4.021034.

[11] Y. T. Chew *et al.*, "Ultraprecise holographic optical tweezer array," *Phys. Rev. A* 110(5), 053518, 2024. https://doi.org/10.1103/PhysRevA.110.053518.

[12] S. Ebadi *et al.*, "Quantum phases of matter on a 256-atom programmable quantum simulator," *Nature* 595, 227–232, 2021. https://doi.org/10.1038/s41586-021-03582-4.

[13] H. J. Manetsch, G. Nomura, E. Bataille, X. Lv, K. H. Leung and M. Endres, "A tweezer array with 6,100 highly coherent atomic qubits," *Nature* 647, 60–67, 2025. https://doi.org/10.1038/s41586-025-09641-4.

[14] T. Harte, G. D. Bruce, J. Keeling and D. Cassettari, "Conjugate gradient minimisation approach to generating holographic traps for ultracold atoms," *Opt. Express* 22(22), 26548, 2014. https://doi.org/10.1364/OE.22.026548.

[15] D. Bowman *et al.*, "High-fidelity phase and amplitude control of phase-only computer generated holograms using conjugate gradient minimisation," *Opt. Express* 25(10), 11692, 2017. https://doi.org/10.1364/OE.25.011692.

[16] P. Chakravarthula, Y. Peng, J. Kollin, H. Fuchs and F. Heide, "Wirtinger holography for near-eye displays," *ACM Trans. Graph.* 38(6), 1–13, 2019. https://doi.org/10.1145/3355089.3356539.

[17] P. Schroff, A. La Rooij, E. Haller and S. Kuhr, "Accurate holographic light potentials using pixel crosstalk modelling," *Sci. Rep.* 13, 3252, 2023. https://doi.org/10.1038/s41598-023-30296-6.

[18] M. Pasienski and B. DeMarco, "A high-accuracy algorithm for designing arbitrary holographic atom traps," *Opt. Express* 16(3), 2176, 2008. https://doi.org/10.1364/OE.16.002176.

[19] N. Holland, D. Stuart, O. Barter and A. Kuhn, "Benchmarking modern algorithms to holographically create optical tweezers for laser-cooled atoms," *J. Mod. Opt.* 65(18), 2133–2141, 2018 (preprint title "Efficient and fast algorithms to generate holograms for optical tweezers," arXiv:1711.07845). https://doi.org/10.1080/09500340.2018.1499978.

[20] H. Swan, A. Torchylo, M. J. Van de Graaff, J. Rudolph and J. M. Hogan, "High-fidelity holographic beam shaping with optimal transport and phase diversity," *Opt. Express* 33(3), 6290–6303, 2025. https://doi.org/10.1364/OE.540901.

[21] A. Torchylo, H. Swan, L. Tellez and J. M. Hogan, "A fast, large-scale optimal transport algorithm for holographic beam shaping," *Opt. Express* 34(4), 6796–6803, 2026. https://doi.org/10.1364/OE.588337.

[22] M. H. Eybposh, N. W. Caira, M. Atisa, P. Chakravarthula and N. C. Pégard, "DeepCGH: 3D computer-generated holography using deep learning," *Opt. Express* 28(18), 26636, 2020. https://doi.org/10.1364/OE.399624.

[23] L. Shi, B. Li, C. Kim, P. Kellnhofer and W. Matusik, "Towards real-time photorealistic 3D holography with deep neural networks," *Nature* 591, 234–239, 2021. https://doi.org/10.1038/s41586-020-03152-0.

[24] Y. Peng, S. Choi, N. Padmanaban and G. Wetzstein, "Neural holography with camera-in-the-loop training," *ACM Trans. Graph.* 39(6), 1–14, 2020. https://doi.org/10.1145/3414685.3417802.

[25] Z. Ren *et al.*, "Creation of a tweezer array for cold atoms utilizing a generative neural network," *APL Quantum* 1(4), 046111, 2024. https://doi.org/10.1063/5.0219170.

[26] R. Sambharya, G. Hall, B. Amos and B. Stellato, "Learning to warm-start fixed-point optimization algorithms," arXiv:2309.07835, 2023 (preprint). https://arxiv.org/abs/2309.07835.

[27] J. Lee, Y. Lee, J. Kim, A. R. Kosiorek, S. Choi and Y. W. Teh, "Set Transformer: a framework for attention-based permutation-invariant neural networks," International Conference on Machine Learning (ICML), 2019; arXiv:1810.00825. https://arxiv.org/abs/1810.00825.

[28] M. Zaheer, S. Kottur, S. Ravanbakhsh, B. Poczos, R. Salakhutdinov and A. Smola, "Deep Sets," Neural Information Processing Systems (NIPS), 2017; arXiv:1703.06114. https://arxiv.org/abs/1703.06114.

[29] M. Tancik *et al.*, "Fourier features let networks learn high frequency functions in low dimensional domains," arXiv:2006.10739, 2020 (preprint). https://arxiv.org/abs/2006.10739.

[30] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," International Conference on Learning Representations (ICLR), 2019; arXiv:1711.05101. https://arxiv.org/abs/1711.05101.

[31] B. Efron, "Bootstrap methods: another look at the jackknife," *Ann. Statist.* 7(1), 1–26, 1979. https://doi.org/10.1214/aos/1176344552.

[32] R. Grimm, M. Weidemüller and Y. B. Ovchinnikov, "Optical dipole traps for neutral atoms," *Adv. At. Mol. Opt. Phys.* 42, 95–170, 2000. https://doi.org/10.1016/S1049-250X(08)60186-X.